

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

REPUBLIC OF CAMEROON

PEACE-WORK-FATHERLAND



UNIVERSITY OF BUEA

P.O. Box 63,
Buea, South West Region
CAMEROON

Tel: (237) 3332 21 34/3332 26 90

Fax: (237) 3332 22 72



FACULTY OF ENGINEERING AND TECHNOLOGY(FET)

CrowdSenseNet

Design and Implementation of a Crowdsensed Drive Test

Mobile Application for Coverage Prediction

A comprehensive mobile network **crowdsensing platform** designed to collect, process, and visualize cellular network measurements through an Android application, secure backend, and administrator dashboard for coverage analysis...

By:

Group 13

For : CEF 440

Course Instructor:

Dr Nkameni Valery

University of Buea

INTERNET PROGRAMMING
AND MOBILE PROGRAMMING

2025/2026 Academic Year

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Group 13 Members

Imele Azafa Jose de Kajos FE23A068
Ayuk Amelia Agbor Oroch FE23A020
Ngome Fernandez Manyaka FE23A111
Tadgeu Ngnintedem Aubry-Junex FE23A148
Bessong Takang Tabot Willy FE23A025

Table of Contents

Table of Contents	3
Introduction.....	6
Structure of This Report	7
Chapter 1: The Mobile Application Development Landscape.....	8
1.1 Types of Mobile Applications	8
Native Applications	8
Progressive Web Apps.....	8
Hybrid Applications.....	8
1.2 Mobile App Programming Languages	9
1.3 Mobile App Development Frameworks	10
1.4 Mobile Application Architectures and Design Patterns	10
1.5 Collecting and Analysing User Requirements	11
1.6 Estimating Mobile App Development Cost	12
Chapter 2: Requirement Gathering.....	13
2.1 Stakeholder Identification	13
2.2 Requirement Gathering Techniques	14
Interviews with MTN Network Engineers.....	14
Surveys of Smartphone Users.....	15
Brainstorming	15
Reverse Engineering Existing Applications	15
2.3 Data Gathering Strategy	16
2.4 Data Cleaning	16
2.5 User Assessment and User Reluctance.....	17
Chapter 3: Requirement Analysis	18
3.1 Requirement Review	18
Technical Feasibility.....	18
3.2 Inconsistencies, Ambiguities, and Missing Requirements	19
3.3 Requirement Prioritisation	19
3.4 Requirement Classification	20
3.5 Software Requirements Specification Summary.....	21
3.6 Validation with Stakeholders	21

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Chapter 4: System Modelling and Design	22
4.1 Context Diagram	22
4.2 Data Flow Diagram	23
4.3 Use Case Diagram	24
4.4 Sequence Diagram: Core Data Collection Scenario.....	25
4.5 Class Diagram	26
4.6 Deployment Diagram	27
Chapter 5: UI Design and Implementation	28
5.1 App Identity and Branding	28
5.2 Visual Screen Design	29
Splash Screen.....	29
Home Screen.....	30
Coverage Map Screen.....	31
History Screen.....	32
Settings Screen.....	33
5.3 Frontend Implementation	33
Chapter 6: Backend Implementation and Database Development	35
6.1 Data Elements Analysis.....	35
6.2 Conceptual Design	36
6.3 Entity Relationship Design.....	37
6.4 Backend Implementation.....	38
6.5 Database Implementation	40
6.6 Connecting the Database to the Backend	40
6.7 Testing and Validation	41
Chapter 7: The Administrator and Network Engineer Dashboard	44
7.1 Purpose and Position in the System	44
7.2 Architecture Extension	44
7.3 Administrator Login	45
7.4 Dashboard Overview	46
7.5 Live Data Feed	46
7.6 Session and Reading Explorer.....	47
7.7 Coverage Map and Heatmap	48
7.8 Device and Session Management.....	49

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

7.9 Reports and Export	49
7.10 Data Flow: From a User's Phone to the Admin Screen	50
7.11 Access Control and Privacy Safeguards	51
Conclusion	52
References	53

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Introduction

Mobile network coverage in Cameroon is inconsistent. Users regularly experience weak signals, slow data speeds, and complete dead zones, not only in remote areas but also inside buildings and in busy urban neighbourhoods. The traditional way of measuring coverage, known as drive testing, involves sending engineers out with specialised equipment to record signal quality along roads. It works reasonably well on main routes, but it is expensive, slow, and cannot easily reach the places where problems are worst: indoors, in rural communities, or anywhere off the main road network.

This report presents CrowdSenseNet, a crowdsensed drive test mobile application that turns ordinary Android smartphones into passive network measurement devices. Instead of relying on a small team of engineers with specialised equipment, the system collects signal readings automatically from everyday users as they go about their lives. Combined across many users and locations, this crowdsourced data produces a much richer and more representative picture of how the network actually performs than traditional drive testing alone.

The project was carried out by Group 13 as the practical component of CEF 440, Internet Programming and Mobile Programming, and was completed in six stages. Each stage built directly on the work of the one before it, moving from research and requirements through to design, implementation, and backend integration. This document consolidates all six stages into a single report, and adds a seventh section describing the administrator and network engineer dashboard, which is the part of the system that turns individual users' contributions into something telecom operators and researchers can actually use.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Structure of This Report

The report is organised as follows:

- Chapter 1 reviews the mobile application development landscape: the types of mobile apps, the programming languages and frameworks available, common architectural and design patterns, and how requirements engineering and cost estimation are normally approached.
- Chapter 2 documents how the team gathered requirements for CrowdSenseNet specifically, combining stakeholder interviews, a 102 respondent user survey, brainstorming, and reverse engineering of existing apps such as OpenSignal and Network Cell Info Lite.
- Chapter 3 takes the raw requirements from Chapter 2 and puts them through a structured analysis: checking completeness and clarity, resolving inconsistencies, prioritising with MoSCoW, and producing a full Software Requirements Specification.
- Chapter 4 translates the specification into a system design using six UML diagram types: context, data flow, use case, sequence, class, and deployment.
- Chapter 5 documents the app's visual identity, the Figma screen designs, and the Jetpack Compose frontend implementation.
- Chapter 6 covers the backend server, the PostgreSQL database schema, and how the two are connected and tested.
- Chapter 7 is new to this consolidated report. It describes the administrator and network engineer dashboard, the web based system that receives the data users upload from their phones and presents it in a form that operators and researchers can act on.

Every design decision described in this report is traceable back to real evidence: interviews with MTN and Orange network engineers, a user survey completed by 102 smartphone users in Buea, and hands on analysis of existing network monitoring applications. The goal throughout has been to build a system that is technically sound, respectful of user privacy and battery life, and genuinely useful to the people who will operate it.

Chapter 1: The Mobile Application Development Landscape

Before any design work began on CrowdSenseNet, the team reviewed the broader landscape of mobile application development: the types of apps that can be built, the languages and frameworks available for building them, the architectural and design patterns that keep a codebase maintainable, and the general processes used for gathering requirements and estimating cost. This chapter summarises that review, which formed the reference point for every technical decision made later in the project.

1.1 Types of Mobile Applications

Mobile applications fall into three broad categories, each with different trade-offs between performance, cost, and reach.

Native Applications

Native apps are built specifically for one platform, using Swift or Objective-C for iOS and Kotlin or Java for Android. Because they are purpose built for the target operating system, they give the best possible performance and the fullest access to hardware such as the camera, GPS, accelerometer, Bluetooth, and NFC. The cost of this is a separate codebase and development team per platform, longer build times, and higher budgets. Examples include Spotify, WhatsApp, and Instagram.

Progressive Web Apps

Progressive Web Apps, or PWAs, are built with standard web technologies (HTML5, CSS3, and JavaScript) and deliver an app like experience directly through the browser, without requiring installation from an app store. They use service workers for offline support and can be added to a device home screen. They are the cheapest and fastest option to build, but they have limited access to native device features and generally weaker performance than a native app. Examples include Twitter Lite and the Starbucks PWA.

Hybrid Applications

Hybrid apps combine both worlds: a web application built with HTML, CSS, and JavaScript, wrapped inside a native container using tools such as Cordova or Capacitor. A single codebase can then be distributed through both the Apple App Store and Google Play. Performance sits between native and pure web apps, and native features are reached through bridge plugins rather than direct access.

Feature	Native	PWA	Hybrid
Performance	Excellent, optimised for the platform	Good, runs in the browser	Moderate, depends on the WebView
Device access	Full hardware access	Limited (no Bluetooth, NFC)	Moderate, via plugins
App store required	Yes	No	Yes

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Feature	Native	PWA	Hybrid
Development cost	High, separate codebase per platform	Low, single web codebase	Moderate, single codebase
Time to market	Slow	Fast	Moderate

Table 1.1: Comparison of native, progressive web, and hybrid mobile applications.

CrowdSenseNet was built as a native Android application. This choice reflects the project's requirement for continuous, low level access to the TelephonyManager and location APIs, something that neither a PWA nor a hybrid app could provide with the same reliability or battery efficiency.

1.2 Mobile App Programming Languages

The language chosen for a mobile project affects development speed, runtime performance, the pool of available talent, and long term maintainability. The languages most relevant to this project are summarised below.

Language	Platform	Performance	Notes
Swift	iOS / Apple	Excellent	Apple's modern, type safe, compiled language for iOS and macOS.
Kotlin	Android	Excellent	Google's preferred Android language since 2017; null safe and fully interoperable with Java.
Java	Android	Good	The original Android language; still widely used for legacy codebases.
Dart	Flutter	Very good (AOT compiled)	Compiles to native ARM code; used almost exclusively with Flutter.
JavaScript / TypeScript	Cross platform	Moderate	Powers React Native and Ionic; TypeScript adds static typing.
C#	Xamarin / .NET MAUI	Good (AOT via Mono)	Best suited to teams already using the Microsoft stack.

Table 1.2: Comparison of major mobile programming languages.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

CrowdSenseNet's Android client is written in Kotlin. Kotlin's null safety, coroutine support for asynchronous background sensing, and full interoperability with the Android Telephony and Location APIs made it the clear choice for a data collection app that needs to run reliably in the background.

1.3 Mobile App Development Frameworks

Four frameworks dominate cross platform mobile development: React Native, Flutter, Ionic, and Xamarin / .NET MAUI. Each was evaluated for this project.

Framework	Language	Performance	Best suited to
React Native	JavaScript / TypeScript	Near native (bridge or JSI)	Teams with web experience wanting native like results quickly
Flutter	Dart	Excellent, own rendering engine	Pixel perfect custom UI and consistent cross platform behaviour
Ionic	HTML, CSS, JavaScript	Moderate, WebView based	Rapid MVPs and content centred apps
Xamarin / .NET MAUI	C#	Good, AOT via Mono	Enterprise teams already using the Microsoft stack

Table 1.3: Comparison of the major cross platform frameworks.

Although CrowdSenseNet is a native Kotlin application rather than a cross platform build, this review directly influenced the choice of MVVM as the architectural pattern (Section 1.4) and Jetpack Compose as the UI toolkit, since these are the same patterns Flutter and modern React Native encourage.

1.4 Mobile Application Architectures and Design Patterns

Architecture defines how an application is structured internally: how data flows, how the UI updates, and how components communicate. Without deliberate architectural planning, an app becomes progressively harder to test, debug, and extend, which is a common failure mode in student and early stage commercial projects alike.

Pattern	Testability	Complexity	Typical use
MVC	Moderate	Low	Small, simple apps; early iOS UIKit development
MVP	High	Moderate	Medium complexity Android apps

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Pattern	Testability	Complexity	Typical use
MVVM	High	Moderate	Medium to large apps with complex UI state; the modern Android and iOS standard
Clean Architecture	Excellent	High	Large enterprise applications
BLoC	Excellent	Moderate to high	Complex Flutter applications

Table 1.4: Comparison of common mobile architectural patterns.

CrowdSenseNet follows the Model View ViewModel (MVVM) pattern throughout. The View (the Jetpack Compose screens) observes a ViewModel, which exposes state through Kotlin StateFlow objects and holds no direct reference to the UI. The Model layer, made up of repositories and local and remote data sources, is completely decoupled from the presentation layer. This keeps the app testable and makes it straightforward for different team members to work on different screens without conflicts, which mattered given the project's five person team.

A handful of classic object oriented design patterns are also used throughout the codebase: the Repository pattern to give each ViewModel a single, consistent source of truth regardless of whether data comes from the local database or the backend; the Observer pattern, which underlies Android's LiveData and StateFlow and drives the reactive UI; the Singleton pattern for shared services such as the local Room database instance; and the Strategy pattern for interchangeable upload behaviour (WiFi only versus WiFi and mobile data).

1.5 Collecting and Analysing User Requirements

Requirements engineering is the systematic discipline of identifying, documenting, validating, and managing what a system needs to do. It matters because a requirement discovered late in development is exponentially more expensive to fix than one caught during planning.

Requirements are generally split into four types: functional requirements (what the system must do), non-functional requirements (how well it must do it, covering speed, security, scalability, and reliability), business requirements (how the system serves organisational goals), and constraints (platform, regulatory, and budget limits).

A typical requirements gathering process moves through five steps: defining the app's idea and purpose, identifying and engaging stakeholders, running a market and competitor analysis, building user personas and scenarios, and finally gathering the functional and non-functional requirements themselves, usually prioritised with the MoSCoW method (Must have, Should have, Could have, Won't have). The most common elicitation techniques are interviews, surveys, contextual observation, focus groups, workshops, user stories, and prototyping. Chapter 2 of this report shows exactly how these techniques were applied to CrowdSenseNet.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

1.6 Estimating Mobile App Development Cost

Cost estimation is critical for project planning and stakeholder alignment. The main cost drivers are app complexity and feature count, platform choice (native for both iOS and Android roughly doubles cost compared with a single cross platform codebase), UI and UX design effort, backend and API integration work, and team location, since hourly rates vary from roughly twenty dollars an hour in parts of Asia to over one hundred dollars an hour in North America.

App type	Complexity	Estimated cost (USD)	Typical timeline
Simple	3 to 5 static screens, no backend	\$5,000 to \$30,000	1 to 3 months
Medium	Login, database, APIs, 10 to 20 screens	\$30,000 to \$100,000	3 to 6 months
Complex	Real time features, AI, payments, 25+ screens	\$100,000 to \$300,000+	6 to 12 months
Enterprise	Custom backend, high security, deep integrations	\$500,000+	12 to 24+ months

Table 1.5: Typical mobile app development cost ranges by complexity tier.

On this scale, CrowdSenseNet sits at the boundary between the medium and complex tiers: it involves user facing screens, offline local storage, a REST backend, a relational database, and a machine learning prediction layer, but with a single platform target and no payment processing. A simple baseline formula for cost is total development hours multiplied by hourly rate, refined in practice with Story Point estimation for Agile teams, Three Point (PERT) estimation where there is significant uncertainty, and Function Point Analysis for large, well specified enterprise projects. A contingency buffer of fifteen to twenty five percent is standard practice to absorb unknowns and scope changes, and ongoing maintenance typically costs fifteen to twenty percent of the initial build cost per year.

Chapter 2: Requirement Gathering

With the general landscape reviewed, the team turned to gathering requirements specific to CrowdSenseNet. This chapter documents who the stakeholders are, the four techniques used to gather their requirements, how the resulting data was collected and organised, and what it revealed about user willingness to participate in a crowdsensing system.

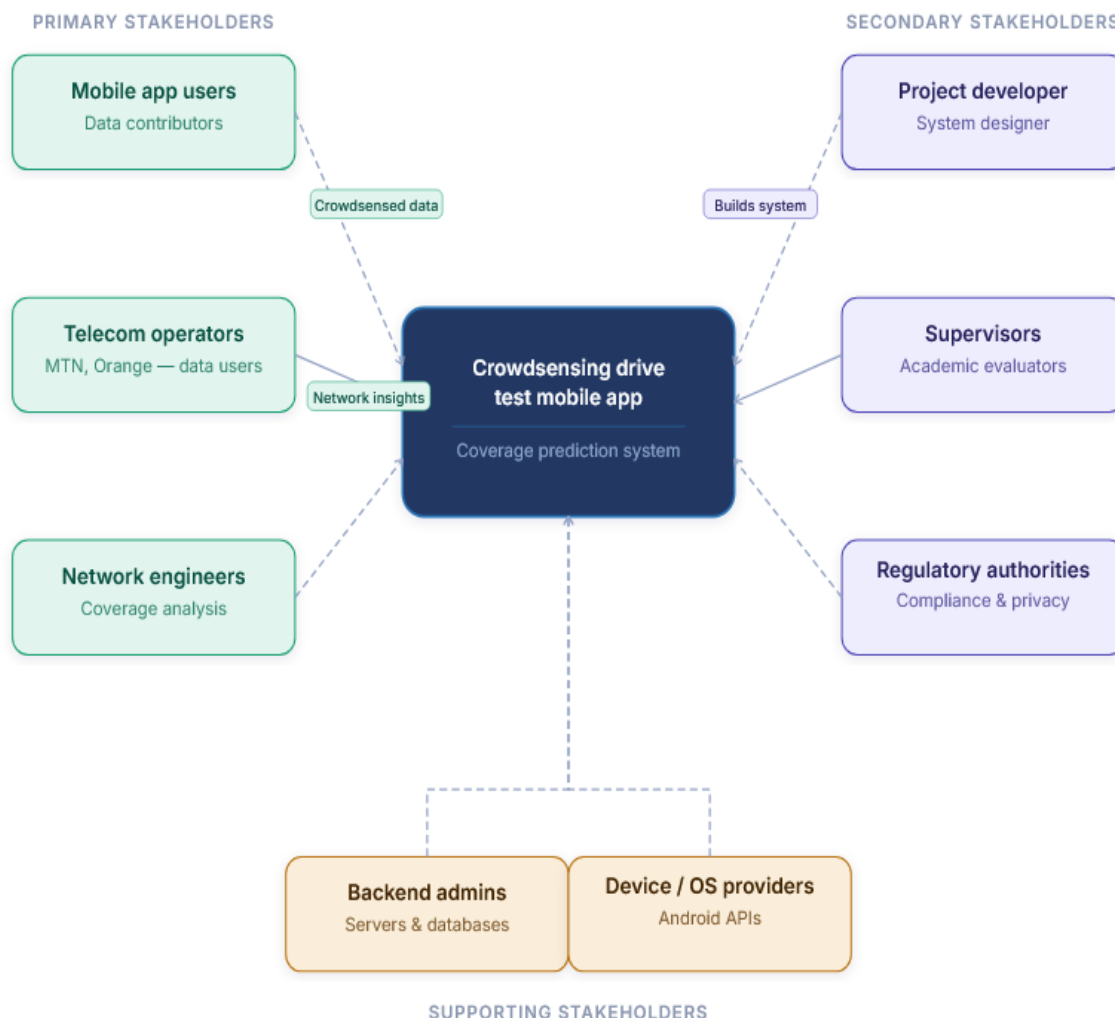
2.1 Stakeholder Identification

Stakeholder identification is the process of identifying everyone who is involved in, affected by, or has an interest in the proposed system. For CrowdSenseNet, stakeholders fall into three groups: primary, secondary, and supporting.

Stakeholder	Role	Primary interest
Mobile application users	Generate network measurement data during normal phone use	Minimal battery and data use; privacy of location data
Telecom network operators (MTN, Orange)	Use collected data to analyse and improve network performance	Accurate, large scale coverage data at lower cost than drive testing
Network engineers	Interpret coverage maps and recommend improvements	High quality, geo-referenced data and visualisation tools
Project developers	Design and build the mobile app and backend systems	Successful, working system that meets requirements
Academic supervisors	Guide and evaluate the project	Sound methodology and clear documentation
Regulatory authorities	Oversee data privacy and fair use of network data	Compliance with data protection frameworks
Backend system administrators	Maintain servers, databases, and uptime	System stability and secure data handling
Device manufacturers / OS providers	Provide the APIs used to access network and location data	Correct use of system APIs and battery efficient background processing

Table 2.1: CrowdSenseNet stakeholders and their primary interests.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING



2.2 Requirement Gathering Techniques

Four complementary techniques were used, each chosen to surface a different layer of information: expert insight from network professionals, real user experience at scale, creative synthesis of everything learned, and a benchmark against what already exists in the market.

Interviews with MTN Network Engineers

MTN was chosen as the interview target because it is the dominant operator in the region, covering roughly seventy one percent of surveyed users. A set of semi-structured questions was prepared covering the key parameters used to measure network performance, the most common problems users report, and whether crowdsourced data could complement MTN's existing monitoring methods. Several of the more technical questions, particularly those touching internal coverage maps and signal thresholds, went unanswered because MTN staff considered that information commercially sensitive and had not received formal authorisation to share it. Even so, the interviews confirmed that RSRP, throughput, and latency are the parameters that matter most, and that coverage is most problematic in rural areas and dense indoor environments.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Surveys of Smartphone Users

Because the interviews did not give full access to technical network data, a survey became the primary tool for gathering user-side requirements. It was distributed through Google Forms and collected 102 responses from a mix of students, working adults, and residents across areas with varying levels of coverage.

Finding	Detail
Overall network quality rating	Average of 2.6 out of 5, below the midpoint
Where poor coverage is felt most	53% indoors; 45% in rural or remote areas
Most reported problem	Slow internet speed, reported by 87% of respondents
Willingness to share data	Majority open to it, but only under specific conditions
Top condition for sharing	56 respondents required full anonymity
Second condition for sharing	37 respondents required minimal battery drain
Preferred collection timing	Most users prefer collection only while the app is open, or fully user controlled

Table 2.2: Key findings from the 102 respondent smartphone user survey.

Each of these findings was translated directly into a system requirement. Below average network ratings became a requirement for real time performance monitoring; the strong preference for indoor and rural data became a requirement to support collection in low connectivity environments; and the conditions attached to data sharing became the non-negotiable privacy and battery requirements that shaped the rest of the project.

Brainstorming

With the interview and survey data in hand, the team held structured brainstorming sessions to bridge the gap between what had been learned and what the app should actually do. Ideas were grouped into four categories: data collection (RSRP, SINR, RSSI, GPS tagging, operator and network type detection), user experience (a visible on and off toggle, a WiFi only upload setting, real time metric display), data handling (offline first local storage, automatic upload once connectivity returns, full anonymisation before storage), and backend and analysis (a REST API, map based visualisation, and a foundation for future machine learning prediction).

Reverse Engineering Existing Applications

Two established network monitoring apps, OpenSignal and Network Cell Info Lite, were installed and used on Android devices to see what already works well and where the gaps are.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Feature	OpenSignal	Network Cell Info Lite
Real time signal display	Yes	Yes, more detailed
Speed test	Yes	No
Map based coverage view	Yes	Basic only
Crowdsourced data collection	Yes, automatic	No
Offline data storage	No	No
User friendly interface	Moderate	No, highly technical

Table 2.3: Feature comparison of OpenSignal and Network Cell Info Lite.

Neither app stores data offline when there is no connection, neither predicts future coverage, and OpenSignal's crowdsourcing happens without much transparency to the user. These three gaps, offline first storage, a foundation for prediction, and transparent user control, became defining requirements for CrowdSenseNet.

2.3 Data Gathering Strategy

Data gathering operated on two levels: primary data collected directly from stakeholders, and secondary data drawn from established technical sources. Primary sources included MNO network engineers (semi-structured interviews), drive test engineers (structured interviews and document review), and general smartphone users in Buea (online and paper surveys). Secondary sources included the 3GPP technical standards that define RSRP, RSRQ, and SINR measurement ranges, ITU-D reports on mobile broadband in Sub-Saharan Africa, and the Android Telephony Manager API documentation, which set the practical boundaries of what data the app could actually collect.

Two findings from the MNO engineer questionnaires shaped core technical parameters directly: the industry consensus that a coverage hole is any location with RSRP below -110 dBm, and a minimum acceptable sampling interval of five seconds, below which professional tools rarely sample.

2.4 Data Cleaning

The raw survey data, one hundred usable responses after an initial duplicate check, was reviewed before any analysis was carried out. Cleaning addressed five goals: accuracy, completeness, consistency, validity, and normalisation. Duplicate submissions sharing the same email address were reduced to a single entry, completely empty rows were dropped, misplaced answers were corrected, similar free text answers such as "no", "not really", and "none" were merged into one consistent value, and a small number of French language responses were translated for consistency.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Metric	Result after cleaning
Network provider split	MTN 65%, Orange 25%, Camtel 10%
Average network quality rating	2.7 out of 5
Most common issue	Slow internet (60%), weak signal (50%)
Willing to share data	55% yes, 25% no, 20% maybe
Top condition for sharing	Full anonymity (70% of those willing)
Acceptable battery drain	Most users accept 1 to 3 percent

Table 2.4: Summary of the cleaned survey dataset.

2.5 User Assessment and User Reluctance

Even a well built app will fail if users do not trust it enough to install it and keep it running. The survey and general research into user behaviour identified nine recurring reasons people hesitate to use an app like CrowdSenseNet: privacy and data security, battery consumption, excessive data usage, sensitive permission requests, performance issues, storage usage, unfamiliarity or lack of trust in the developer, notification spam, and hidden costs.

Each concern was mapped to a concrete design response. Privacy fears are addressed with a transparent, opt-in consent flow and full anonymisation before any data leaves the device. Battery concerns are addressed with adaptive sampling that pauses collection when the phone is stationary. Data usage concerns are addressed with a WiFi only upload default. Trust is built through university affiliation, a clear privacy policy, and complete user control over when data collection is active. None of these responses were treated as optional extras; they were built into the requirements as non-negotiable design priorities, and they carry through directly into the Software Requirements Specification in the next chapter.

Chapter 3: Requirement Analysis

Chapter 2 produced a large body of raw requirements drawn from interviews, a user survey, brainstorming, and reverse engineering. This chapter describes how that material was reviewed, cleaned up, prioritised, and formalised into a Software Requirements Specification before any design work began.

3.1 Requirement Review

The gathered requirements were reviewed across four dimensions: completeness, clarity, technical feasibility, and dependency relationships. Coverage of RF parameters, privacy, battery efficiency, the on and off toggle, and the WiFi only upload option was found to be solid. Two areas needed more work: some RF parameter requirements had been inferred rather than formally confirmed because MTN kept certain data confidential, and the GPS accuracy threshold existed only as an implied requirement until it was written down explicitly.

A number of requirements were also too vague to be tested. Each ambiguous requirement was rewritten in clear, measurable language.

Original (ambiguous)	Clarified (measurable)
App must use minimal battery	Background sensing must not increase battery drain by more than 5% per hour
Simple, clean interface	No more than 3 taps required to enable or disable data collection
Anonymous data	The app shall not collect name, email, phone number, IMEI, or IP address
Predict coverage holes	A coverage hole is defined as predicted RSRP below -110 dBm
Works offline	Data collection, local storage, and history viewing shall work offline

Table 3.1: Examples of ambiguous requirements clarified into testable statements.

Technical Feasibility

Every major requirement was rated feasible, partially feasible, or not feasible within the project's scope and timeline. Collecting RF parameters through the Android Telephony API, GPS tagging, offline local storage with Room, and multi-operator support were all rated feasible. Map based heatmap visualisation and GeoJSON export were rated partially feasible, since both require backend integration effort beyond the mobile client itself. Machine learning based coverage prediction was rated not feasible in the current phase, since it depends on a live data pipeline that does not yet exist.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

3.2 Inconsistencies, Ambiguities, and Missing Requirements

Four genuine conflicts were found between requirements that had both been validated by real evidence.

- Battery efficiency versus real time collection. Users wanted continuous data collection and minimal battery use at the same time, which is physically contradictory. Resolved with adaptive sampling: readings every ten seconds while the device is moving, with collection pausing automatically after thirty seconds of no movement.
- WiFi only upload versus a live coverage map. Fifty six percent of users wanted uploads restricted to WiFi, but the app also promises a fresh, live map. Resolved by always storing data locally first, defaulting to WiFi only upload, and letting the map refresh whenever new data does arrive at the server.
- Full anonymity versus device consistency for machine learning. Fully anonymous data is hard to reconcile with a model that benefits from linking readings from the same device over time. Resolved with a random anonymous UUID generated on first launch, which allows device level consistency without collecting any personal information.
- Collection timing preference. Roughly even numbers of users wanted app-open-only collection and always-on background collection, so both cannot be a single fixed default. Resolved by making background collection the default with a settings toggle to switch to app-open-only mode.

Beyond the ambiguities already listed in Table 3.1, eleven gaps were identified where important scenarios had simply not been addressed: the minimum supported Android version, what happens when GPS is disabled mid-session, how the app behaves when a user denies a permission, a cap on local storage with automatic cleanup of already-synced readings, retry behaviour when the server is unreachable, rejection of GPS readings that fall outside Cameroon's geographic bounding box, averaging of duplicate readings taken within ten metres of each other, a twelve month data retention policy, a minimum data density of fifty readings per square kilometre before a machine learning prediction can be generated, and a minimum accuracy target of eighty percent for that prediction model.

3.3 Requirement Prioritisation

All requirements were scored on importance and feasibility and sorted using the MoSCoW method.

Priority	Count	Meaning
Must have	5	Non-negotiable; defines the Minimum Viable Product
Should have	4	High value; targeted for the v1.0 release after the MVP core
Could have	3	Useful, deferred if time allows
Won't have (this phase)	3	Important long term but out of scope for now

Table 3.2: MoSCoW prioritisation summary.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

The five Must Have requirements, all high importance and high feasibility, are: collecting RSRP, SINR, RSRQ, and GPS as the foundation of the entire system; full data anonymisation, since fifty six percent of willing respondents made it a hard condition; a five second sampling interval, the industry minimum confirmed by engineers; an on and off toggle for data collection, requested explicitly by thirty six survey respondents; and discarding GPS readings with an accuracy worse than fifteen metres, to keep the coverage predictions geospatially sound.

The Should Have set covers WiFi only upload, battery efficient background sensing, transparent consent prompts, and GeoJSON and CSV export with a heatmap, all of which matter a great deal but involve more implementation complexity than the Must Have set. Could Have items include a real time signal dashboard, a configurable sampling interval, and indoor or outdoor environment tagging. Machine learning based coverage prediction, multi-operator comparative analytics, and iOS support are explicitly Won't Have for this phase, each deferred for a clear reason rather than simply forgotten.

3.4 Requirement Classification

The final set of sixteen requirements splits into eight functional requirements, describing what the system must do, and eight non-functional requirements, describing how well it must do it.

Category	Requirement	Rationale
Privacy	Full data anonymisation before transmission	56% of willing users required it
Performance	Battery efficient background sensing	Battery drain was the #2 reluctance factor (38%)
Usability	Minimal interaction required during collection	Passive collection is the core design principle
Reliability	GPS accuracy threshold of 15 metres or better	Inaccurate GPS makes the coverage map unusable
Data quality	Minimum sampling rate of 5 seconds	Confirmed minimum threshold from engineer interviews
Security	Informed consent before any collection begins	Regulatory and ethical requirement
Scalability	Aggregate data from large numbers of concurrent users	Crowdsensing only works at scale
Compatibility	Android support via the TelephonyManager API	Target users are predominantly Android based

Table 3.3: Non-functional requirements and their evidence based rationale.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

3.5 Software Requirements Specification Summary

All of the analysis above was formalised into a full Software Requirements Specification for CrowdSenseNet. The project scope covers an Android application that passively collects RSRP, RSRQ, and SINR readings to build real time coverage heatmaps and, in a future phase, predict coverage holes using machine learning. iOS support is explicitly out of scope. The MVP covers signal data collection, local offline storage, backend upload, and a basic coverage map.

Constraint	Detail
Minimum Android version	Android 8.0 (API level 26)
Local database	Room, built on SQLite
Remote database	PostgreSQL with the PostGIS spatial extension
Geographic scope	Bounded to Cameroon (latitude 1.65N to 13.08N)
Required permissions	ACCESS_FINE_LOCATION, READ_PHONE_STATE
Minimum data density for ML prediction	50 readings per square kilometre

Table 3.4: Key constraints from the CrowdSenseNet Software Requirements Specification.

3.6 Validation with Stakeholders

Writing down a requirement is not the same as confirming it is correct. Each stakeholder group was validated through a method suited to their concerns: users through follow up surveys and prototype testing of the on and off toggle, network engineers through requirement review meetings confirming that RSRP, SINR, and RSSI matched their own internal standards, regulators through a compliance check of the anonymisation strategy, and academic supervisors through a review of the methodology itself.

Every requirement was run through a five point checklist covering clarity, completeness, feasibility, testability, and stakeholder acceptance before it was treated as final. The outcome confirmed that the anonymity and on and off control requirements are locked in as non-negotiable, that the chosen RF parameters match what MTN and Orange engineers already use internally, and that the functional and non-functional classification follows accepted software engineering methodology. With that confirmation in hand, the team moved into system design, covered in the next chapter.

Chapter 4: System Modelling and Design

With the requirements formalised, the team translated them into a system design using six Unified Modelling Language (UML) diagram types: a context diagram to define the system boundary, a data flow diagram to show how information moves internally, a use case diagram to map user interactions, a sequence diagram to model the core data collection scenario over time, a class diagram describing the static structure of the Android app under the MVVM pattern, and a deployment diagram showing how software components are distributed across physical infrastructure. Diagrams were produced using draw.io and EdrawMax, and every design decision is traceable to a specific requirement from Chapter 3.

4.1 Context Diagram

The context diagram is the highest level view of the system: CrowdSenseNet shown as a single process, surrounded by every external actor that sends it data or receives data from it. Four actors were identified. Users send signal readings, consent confirmations, and on and off toggle commands, and receive coverage feedback and upload status notifications in return. Network engineers at MTN and Orange send map and export requests and receive processed heatmaps and GeoJSON or CSV exports. Regulators send privacy guidelines and receive anonymisation compliance reports. Academic supervisors send validation feedback and receive project documentation.

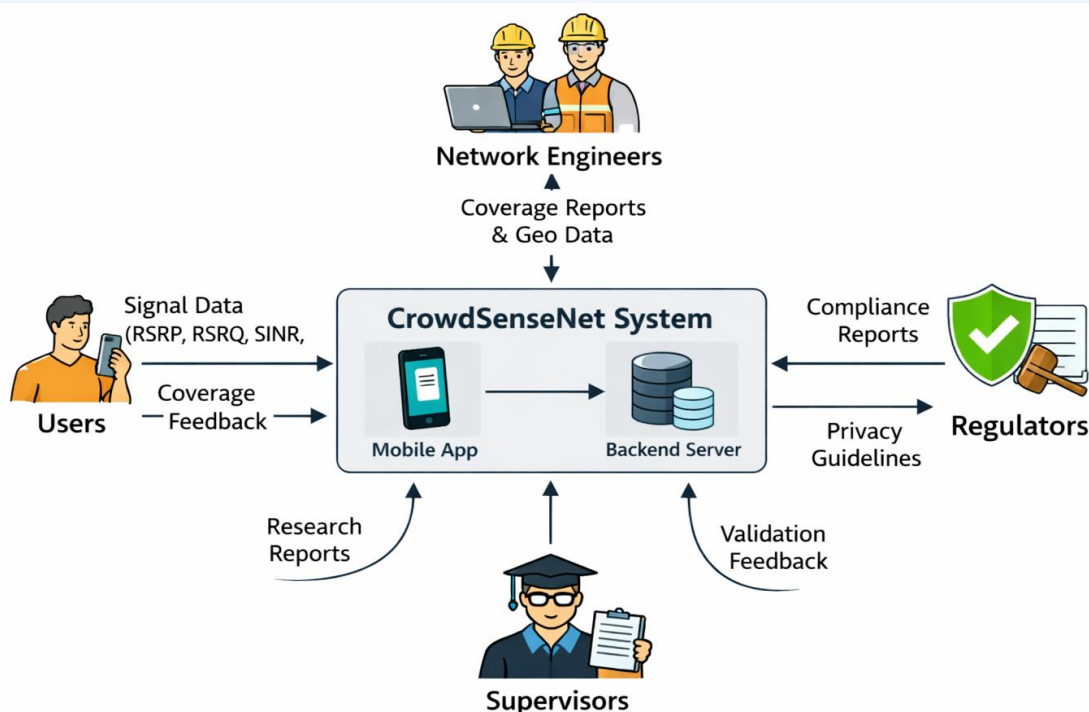


Figure 4.1: Context Diagram for CrowdSenseNet

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

4.2 Data Flow Diagram

The Level 1 data flow diagram opens the single process from the context diagram into seven internal processes: user consent and onboarding, signal data collection, local storage management, data upload and synchronisation, backend validation and storage, coverage map generation, and machine learning coverage prediction, the last of which is modelled but deferred to a future phase.

- Process 1.0, consent and onboarding, generates the anonymous UUID on first launch and stores it in local preferences.
- Process 2.0, signal data collection, reads RSRP, RSRQ, SINR, and RSSI from the TelephonyManager API and GPS coordinates from FusedLocationProvider every ten seconds while the device is moving, discarding any GPS fix worse than fifteen metres.
- Process 3.0, local storage management, writes every valid reading to the Room database immediately, keeping the app fully functional offline and capping local storage at fifty megabytes.
- Process 4.0, upload and synchronisation, monitors connectivity and sends unsynced readings over HTTPS whenever WiFi is available, retrying automatically every thirty minutes on failure.
- Process 5.0, backend validation and storage, checks that incoming coordinates fall inside Cameroon's bounding box, averages duplicate readings within a ten metre radius, and writes valid records to PostgreSQL.
- Process 6.0, coverage map generation, classifies readings into good, average, poor, and coverage hole zones and returns the result as a GeoJSON feature collection.
- Process 7.0, machine learning coverage prediction, is modelled but deferred to a future phase.

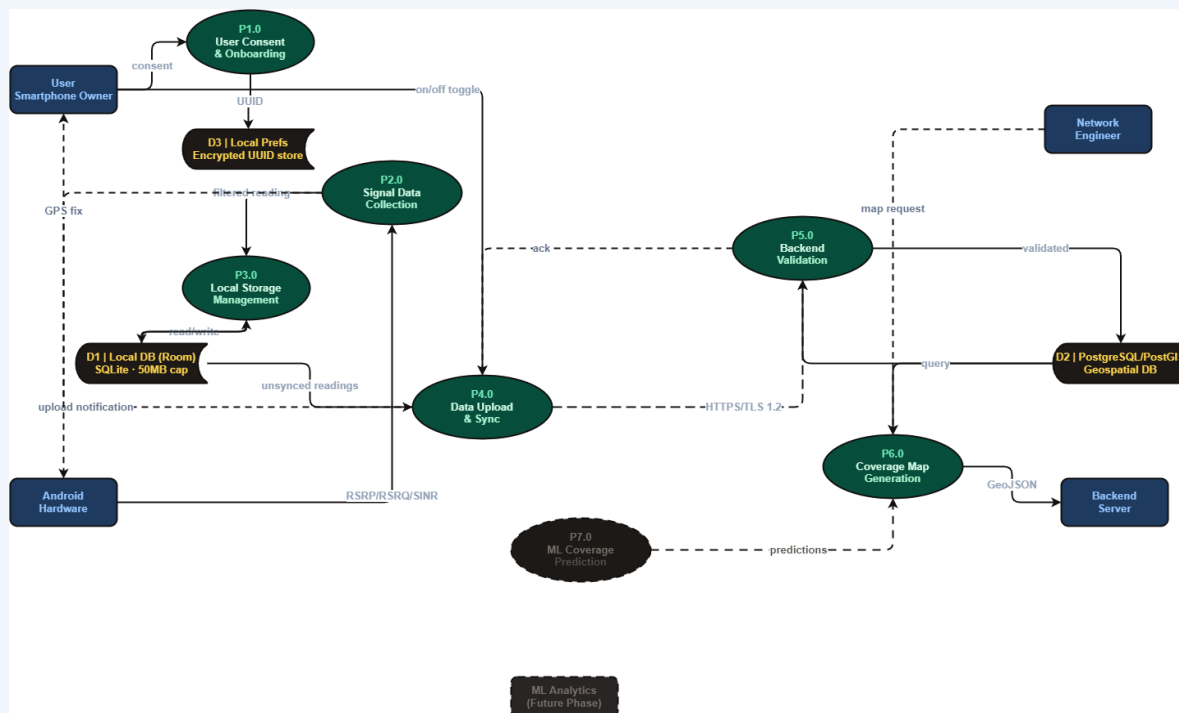


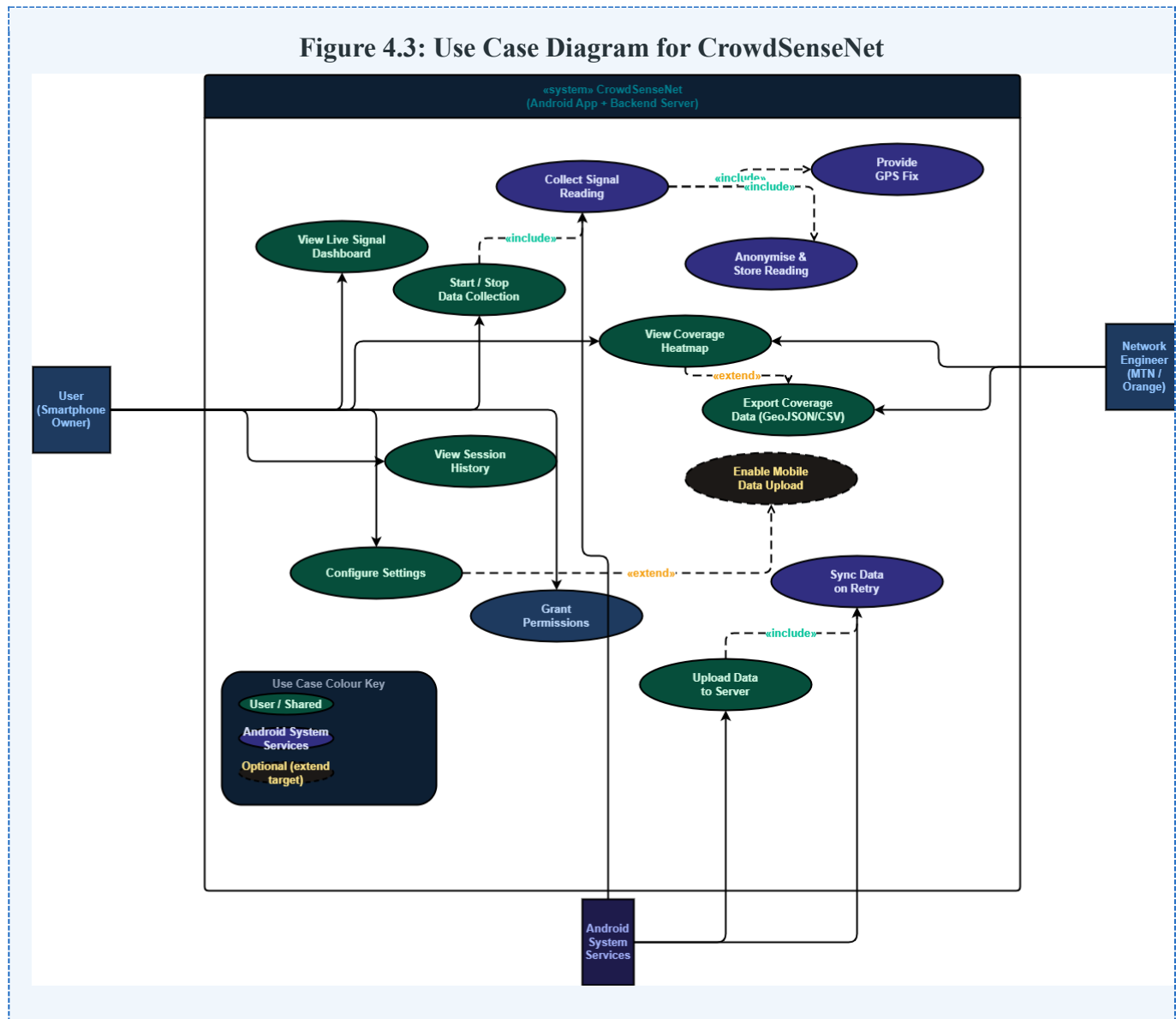
Figure 4.2: Level 1 Data Flow Diagram for CrowdSenseNet

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

4.3 Use Case Diagram

The use case diagram identifies three primary actors, the smartphone user, the network engineer, and the non-human actor Android System Services, and maps out twelve use cases including start and stop data collection, view live signal dashboard, view coverage heatmap, view session history, configure settings, upload data, export coverage data, grant permissions, collect signal reading, provide GPS fix, anonymise and store reading, and sync data on retry. Include relationships capture the fact that starting collection always triggers a reading cycle, every reading always includes a GPS fix, and every reading is always anonymised before storage. Extend relationships capture optional behaviour, such as engineers optionally exporting what they are viewing on the map.

Figure 4.3: Use Case Diagram for CrowdSenseNet



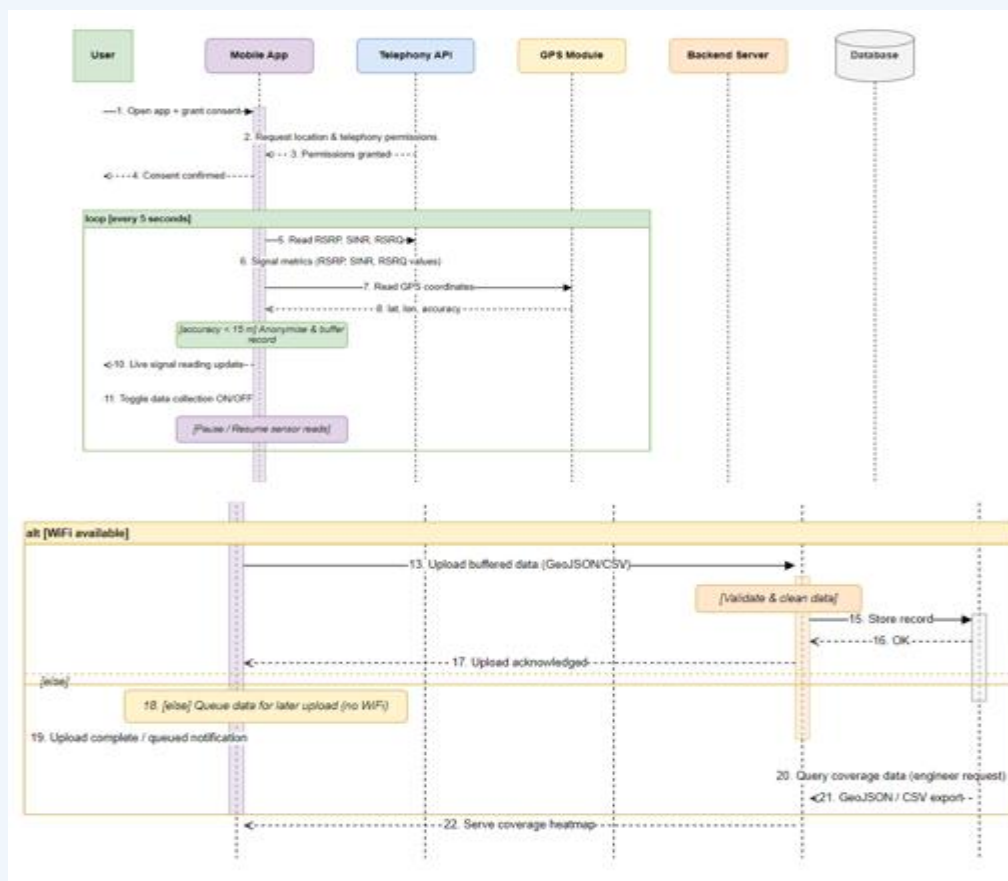
INTERNET PROGRAMMING AND MOBILE PROGRAMMING

4.4 Sequence Diagram: Core Data Collection Scenario

The sequence diagram models the time ordered flow of the core scenario across six lifelines: the User, the Mobile App, the Telephony API, the GPS Module, the Backend Server, and the Database. The scenario runs in four phases.

- Phase 1, app launch and consent: the user grants location and phone state permissions, and the app activates the background sensor loop.
- Phase 2, the data collection loop, repeating every ten seconds while the device is moving: the app reads RSRP, SINR, and RSRQ, requests a GPS fix, discards any fix worse than fifteen metres, anonymises the valid reading on-device, and writes it to the local database, pausing automatically after thirty seconds of no movement.
- Phase 3, upload, modelled as two branches: on WiFi, the buffered batch is sent over HTTPS/TLS and marked as synced on confirmation; without WiFi, or with the WiFi-only restriction active, the upload is deferred and a retry is scheduled for thirty minutes later.
- Phase 4, coverage map serving: when the Map screen opens, the app queries the backend, which queries the database and returns a GeoJSON heatmap for display.

Figure 4.4: Sequence Diagram for the Core Data Collection Scenario

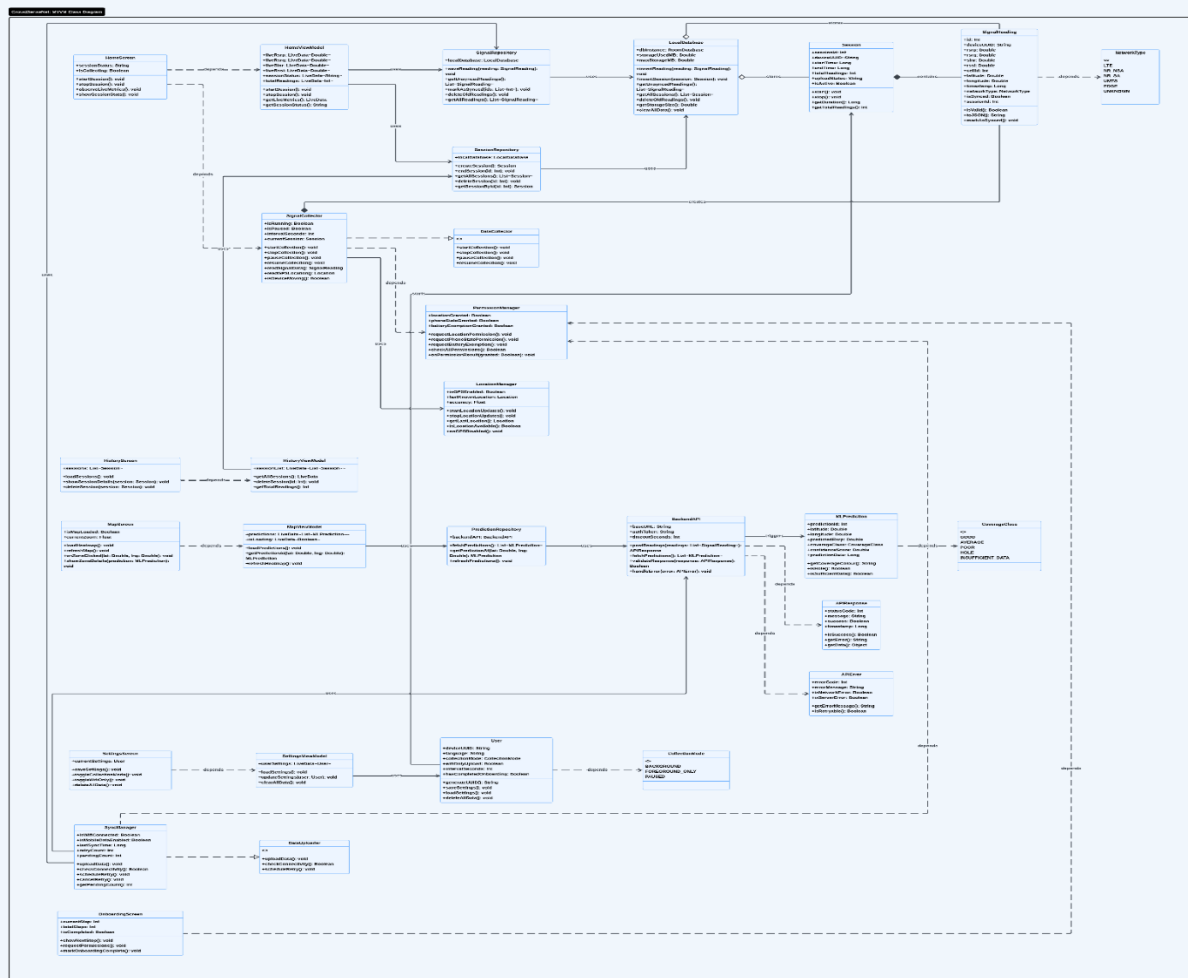


INTERNET PROGRAMMING AND MOBILE PROGRAMMING

4.5 Class Diagram

The Android application's static structure follows MVVM and is organised into seven packages: Presentation (five screen composables), ViewModel (four ViewModels exposing StateFlow to their screens), Domain and Service (the DataCollector and DataUploader interfaces and their SignalCollector, SyncManager, PermissionManager, and LocationManager implementations), Repository (SignalRepository, SessionRepository, PredictionRepository), Data (the SignalReading, Session, User, MLPrediction, and LocalDatabase entities), Network (BackendAPI, APIResponse, APIError), and three enumerations covering coverage class, network type, and collection mode. In total the design covers twenty nine classes, two interfaces, three enumerations, and thirty two relationships, expressed through dependency, association, aggregation, and composition links. A Session, for example, is composed of its SignalReading records: deleting a session cascades to delete every reading that belongs to it.

Figure 4.5: Class Diagram for CrowdSenseNet (MVVM Architecture)

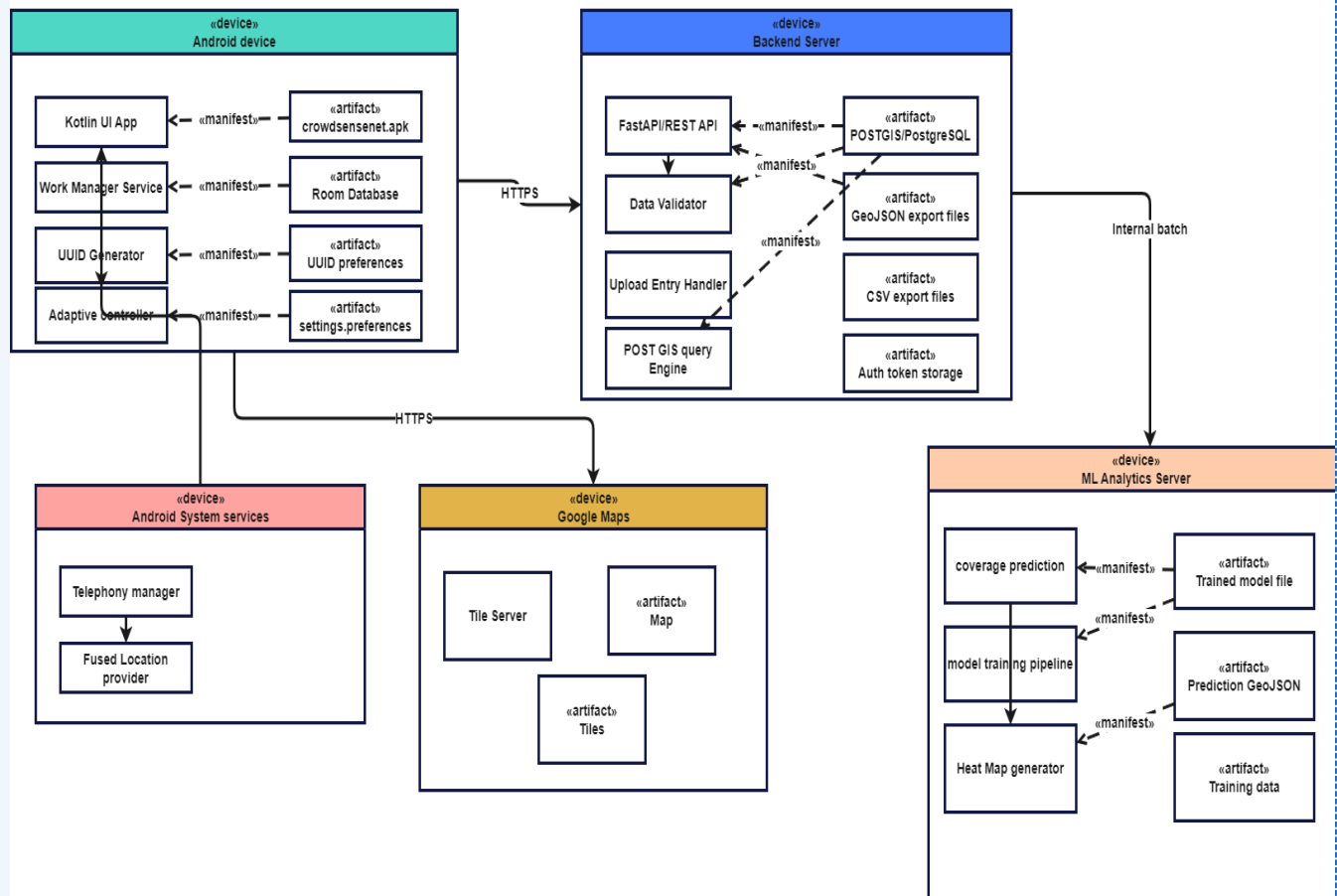


INTERNET PROGRAMMING AND MOBILE PROGRAMMING

4.6 Deployment Diagram

The deployment diagram maps the software onto five physical or virtual nodes across three tiers: the Android Device (running the Kotlin app, the WorkManager background sync service, the adaptive sampling controller, and the Room database), Android System Services (the TelephonyManager and FusedLocationProvider hardware APIs the app consumes), the Backend Server (FastAPI, a data validator, and the PostGIS spatial query engine), the ML Analytics Server (the coverage prediction model, its training pipeline, and a heatmap generator, included here even though it is a future phase item), and the external Google Maps Tile Service that supplies the base map layer. All communication between the Android device and the backend uses HTTPS with TLS 1.2 or higher.

Figure 4.6: Deployment Diagram for CrowdSenseNet



Chapter 5: UI Design and Implementation

This chapter documents how the design work from Chapter 4 was turned into a visual identity, a set of high fidelity screen designs, and a working Android frontend. It covers three linked stages: establishing the app's brand identity, designing every screen in Figma, and implementing those designs in Kotlin using Jetpack Compose.

5.1 App Identity and Branding

The app is named CrowdSenseNet, a name built from three words that describe exactly what it does: Crowd, the community of smartphone users contributing data; Sense, the sensing capability of each phone reading RSRP, RSRQ, SINR, RSSI, and location; and Net, both the cellular network being measured and the network of users collaborating to map it.

The logo combines three visual elements. A map pin, the universal symbol for location, forms the dominant shape, reflecting the fact that every reading the app collects is tied to a GPS coordinate. Inside the pin sits a network mesh, a pattern of connected nodes that represents both the cellular network itself and the machine learning intelligence applied to the collected data. Above the pin sits three concentric signal waves echo the app's three coverage classes: good, average, and poor. The whole mark is rendered in a teal to deep royal blue gradient.

Figure 5.1: Official CrowdSenseNet App Icon and Logo



The primary tagline, "Your phone. Your network. Your map.", speaks directly to the user: the device already in their pocket is all that is needed, the signal quality they already live with is what gets measured, and the resulting map belongs to the whole community that contributed to it.

Colour	Hex code	Role
Deep Royal Blue	#1565C0	Primary: navigation bar, primary buttons, headers
Teal / Cyan	#00BCD4	Secondary: accents, signal indicators, card borders

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Colour	Hex code	Role
White	#FFFFFF	Background: screens, cards, input fields
Dark Slate	#263238	Text: body copy, labels, headings
Signal Green	#4CAF50	Good coverage (RSRP above -80 dBm)
Amber	#FF9800	Average coverage (RSRP -80 to -100 dBm)
Danger Red	#F44336	Coverage hole (RSRP below -110 dBm) and destructive actions
Neutral Grey	#9E9E9E	Insufficient data (fewer than 50 readings per km2)

Table 5.1: CrowdSenseNet brand colour palette.

Typography uses Roboto throughout, Google's default Material Design typeface, at sizes ranging from 32sp for the app name down to 12sp for timestamps and captions. Five design principles guide every screen: Material Design 3 conventions throughout, simplicity first so a non-technical user can start contributing within sixty seconds, a strict three tap rule for reaching any core feature, privacy by design with no login screen and no user profile of any kind, and colour blind accessibility, meaning every coloured coverage zone on the map also carries a text label so the information does not depend on colour perception alone.

5.2 Visual Screen Design

All five screens, Splash, Home, Coverage Map, History, and Settings, were designed at high fidelity in Figma using a 360 by 800 dp Android frame and a shared component library of primary, secondary, and destructive buttons, metric cards, session cards, and a consistent top app bar and bottom navigation bar.

Splash Screen

A simple white screen showing the centred logo, the app name in bold, the tagline, and a small loading indicator while the app initialises its database in the background.

Figure 5.2: Splash Screen Design

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

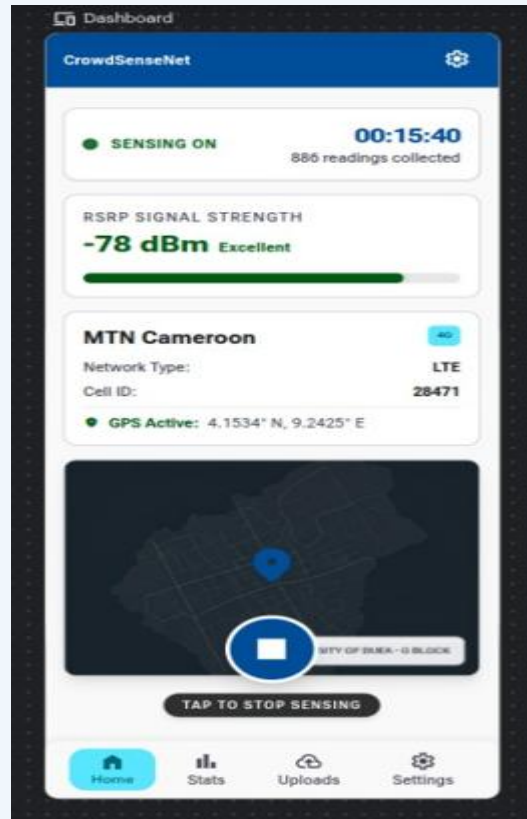


Home Screen

The main control screen, built from a GPS status card, a two by two grid of live metric cards for RSRP, RSRQ, SINR, and RSSI, a session status panel showing duration, reading count, and upload status, and full width Start and Stop buttons in signal green and danger red respectively.

Figure 5.3: Home Screen Design

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

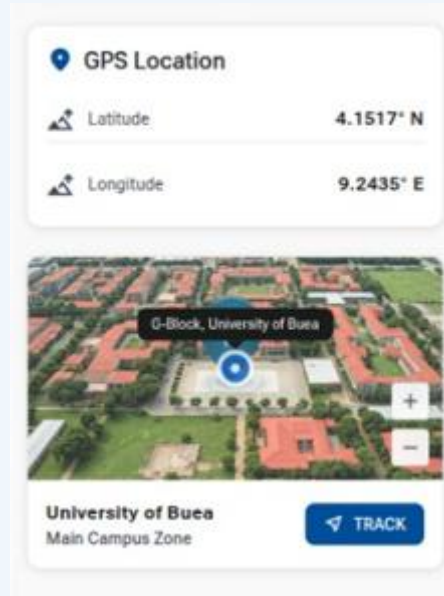


Coverage Map Screen

The most visually complex screen: a search bar, a large map area with semi-transparent coloured polygons showing coverage quality, a horizontal legend bar with the four coverage classes, and an Export Coverage Data button.

Figure 5.4: Coverage Map Screen Design

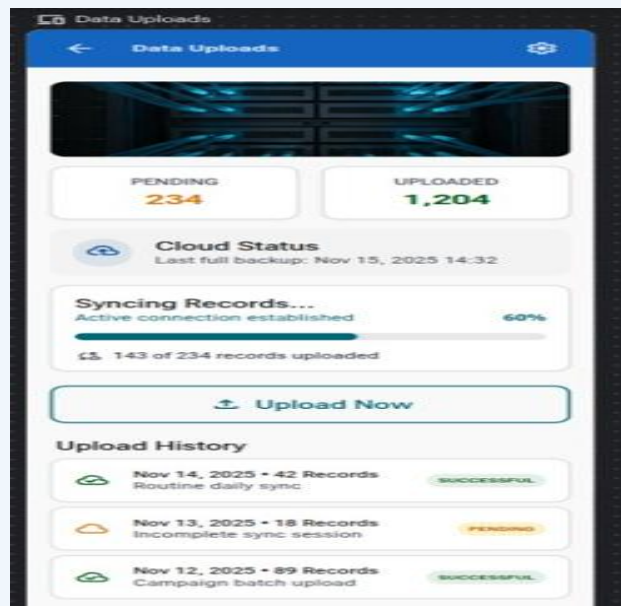
INTERNET PROGRAMMING AND MOBILE PROGRAMMING



History Screen

A reverse chronological, scrollable list of session cards, each showing date and time, duration, reading count, and upload status, with a dedicated empty state shown the first time a user opens the screen before any session has been recorded.

Figure 5.5: History Screen Design

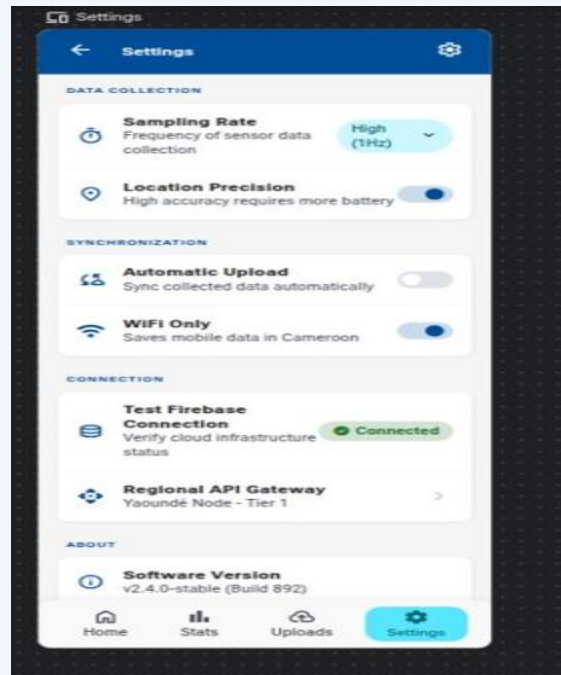


INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Settings Screen

Three grouped cards: language selection (English and French), transmission rules (WiFi only upload and a WiFi enabled toggle), and a storage and administration card holding the Delete All Data action in danger red, with a clear warning that the action is permanent.

Figure 5.6: Settings Screen Design



5.3 Frontend Implementation

Every screen was implemented in Kotlin using Jetpack Compose, following the MVVM pattern established in Chapter 4. Each screen observes a dedicated ViewModel through Kotlin StateFlow, so only the parts of the UI that actually change are recomposed.

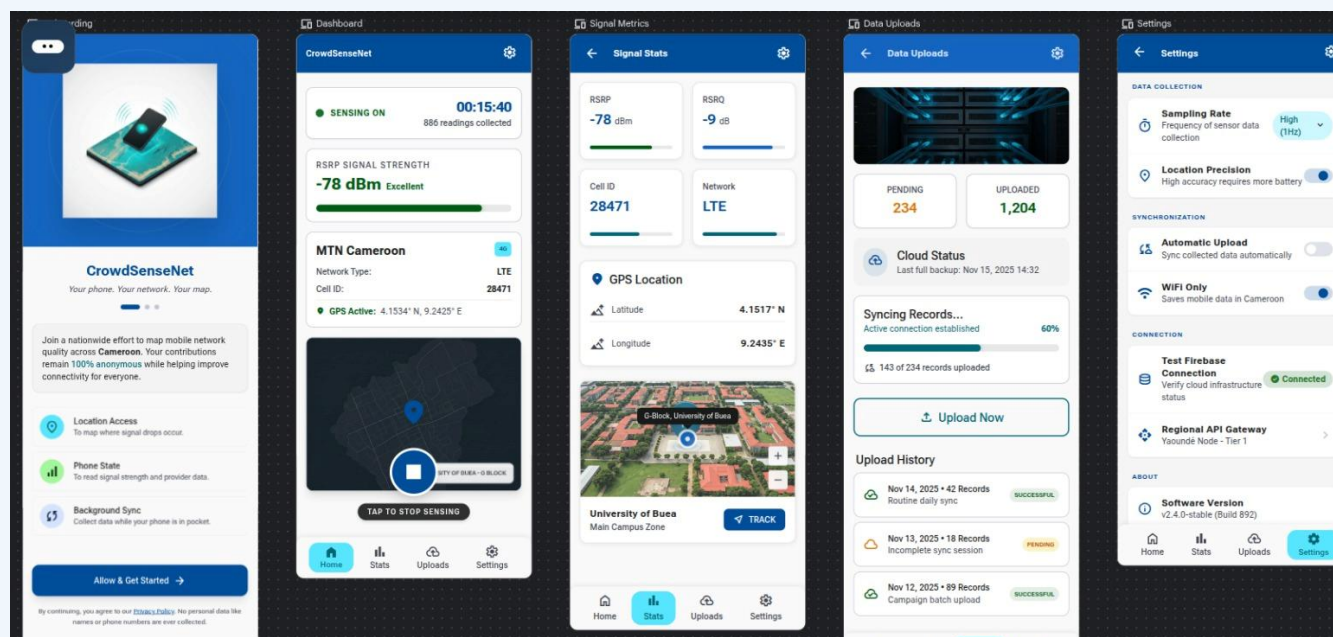
Screen	Key implementation detail
Home	Polls TelephonyManager.getAllCellInfo() every three seconds while a session is active; GPS state driven by a LocationListener with three visual states (active, searching, disabled)
Coverage Map	Google Maps Compose library; polygons drawn per 1km by 1km grid cell at 60% opacity; search uses the Android Geocoder API; export writes a CSV and launches an Android share intent

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Screen	Key implementation detail
Settings	Settings persisted with Android DataStore; language switch updates a bilingual string map without a full app restart; Delete All Data requires an AlertDialog confirmation before the Room database is wiped
History	Room database exposed as a Flow, collected with Paging 3 so the list stays fast even with hundreds of stored sessions; sorted server-side by an ORDER BY clause in the query
Navigation	A single NavGraph.kt sealed class defines all five destinations; the bottom navigation bar observes the current back stack entry to highlight the active tab

Table 5.2: Summary of frontend implementation detail by screen.

Figure 5.7: CrowdSenseNet Running on a Physical Device



With the frontend complete, the client side of CrowdSenseNet was functionally whole: a user could install the app, grant permissions, collect anonymised signal data, view it locally, and upload it whenever a WiFi connection became available. The next chapter turns to the other half of the system, the backend server and database that receive this data and turn it into something a network engineer can act on.

Chapter 6: Backend Implementation and Database Development

The Android client built in Chapter 5 can collect and store data on its own, but a crowdsensing system only becomes useful once readings from many devices are combined. This chapter documents the data elements the system works with, the conceptual design of the three tier architecture, the entity relationship model behind the database, the backend server itself, and how the two are connected and tested.

6.1 Data Elements Analysis

Twenty three distinct data elements were identified across six categories: device data, session data, signal reading data, location data, synchronisation data, and coverage analysis data. Privacy by design shaped every one of them. The Device ID, for example, is a UUID generated on the device itself using Java's `UUID.randomUUID()`, never derived from a hardware identifier such as the IMEI, so it cannot be traced back to a specific person.

Element	Type	Range	Source
RSRP	Float (dBm)	-140 to -44 dBm	TelephonyManager.getAllCellInfo()
RSRQ	Float (dB)	-20 to -3 dB	TelephonyManager.getAllCellInfo()
SINR	Float (dB)	-20 to +30 dB	TelephonyManager.getAllCellInfo()
RSSI	Float (dBm)	-113 to -51 dBm	TelephonyManager.getAllCellInfo()
Latitude / Longitude	Double (degrees)	WGS-84 range	Android LocationManager / GPS
GPS accuracy	Float (m)	3 to 10 m typical	Location.getAccuracy()
Device UUID	UUID string	UUID v4	Generated once, stored in SharedPreferences
Coverage class	Enum	Good / Average / Hole / Insufficient	Derived server-side from average RSRP

Table 6.1: Selected data elements from the full 23-element dictionary.

Coverage class	RSRP threshold	Colour
Good	Above -80 dBm	Signal Green #4CAF50
Average	-80 to -100 dBm	Amber #FF9800
Coverage hole	Below -110 dBm	Danger Red #F44336

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

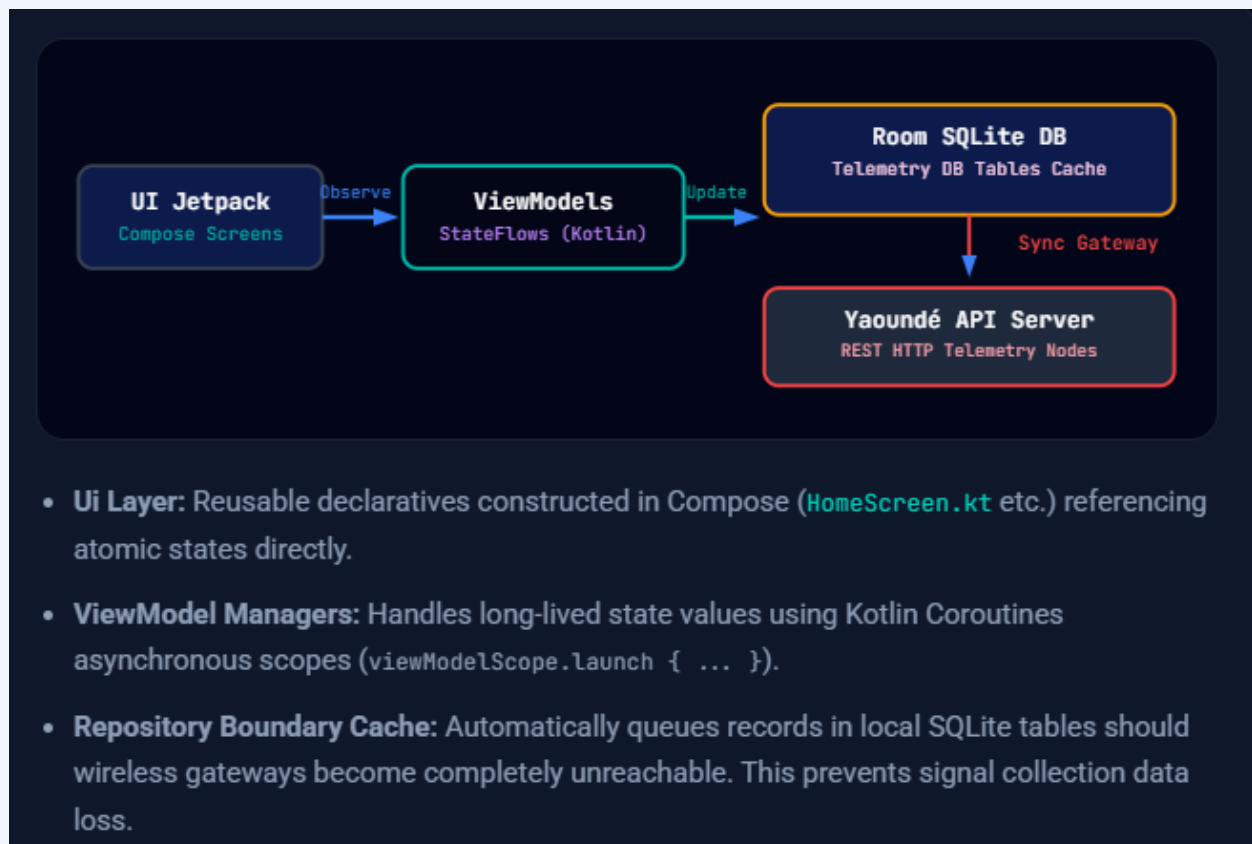
Coverage class	RSRP threshold	Colour
Insufficient data	Fewer than 50 readings per km2	Neutral Grey #9E9E9E

Table 6.2: Coverage classification thresholds used across the mobile app, backend, and admin dashboard.

6.2 Conceptual Design

The system involves two primary actors: the General User, who contributes data anonymously with no account or login required, and the Administrator, a system operator or engineer who accesses the backend dashboard described fully in Chapter 7 to view uploaded data, generate reports, and export it for further analysis. Three components carry the weight of the system. The Android application is the mobile data collection layer, storing readings locally in Room until they can be uploaded. The backend server, built with Node.js and Express following an MVC and Repository pattern, is the processing and routing layer: it validates incoming data, applies coverage classification, and stores results. The PostgreSQL database, extended with PostGIS for spatial queries, is the persistence layer and the single source of truth for all server side data.

Figure 6.1: High Level System Architecture



INTERNET PROGRAMMING AND MOBILE PROGRAMMING

6.3 Entity Relationship Design

The database was modelled with twelve entities across five logical layers, normalised to Third Normal Form so that no data is repeated unnecessarily and every non-key attribute depends only on its own table's primary key. Every table uses a UUID primary key rather than an auto-incrementing integer, because the identifier for a Session or SignalReading is generated on the device at the moment of collection, before it ever reaches the server.

Entity	Purpose
Device	An anonymously registered smartphone participating in data collection
Session	One bounded data collection event, from Start to Stop
SignalReading	One measurement: four signal metrics plus GPS and cell tower context
Location	The GPS coordinate and accuracy for a reading, separated out to avoid redundancy
CellTower	The serving base station: cell ID, operator, and radio technology
AppSettings	Per-device preferences: language, WiFi-only upload, and so on
ExportFile	Metadata for each CSV or GeoJSON export generated from a session
ServerUpload	A record of every upload attempt, successful or failed, for audit and retry logic
CoverageZone	A geographic grid cell classified as good, average, poor, or insufficient data
ZoneReading	Associative entity resolving the many-to-many link between zones and readings
Admin	An authorised dashboard user with hashed credentials and an assigned role
AnalysisReport	A structured coverage report generated by an administrator for a given zone

Table 6.3: The twelve entities in the CrowdSenseNet data model.

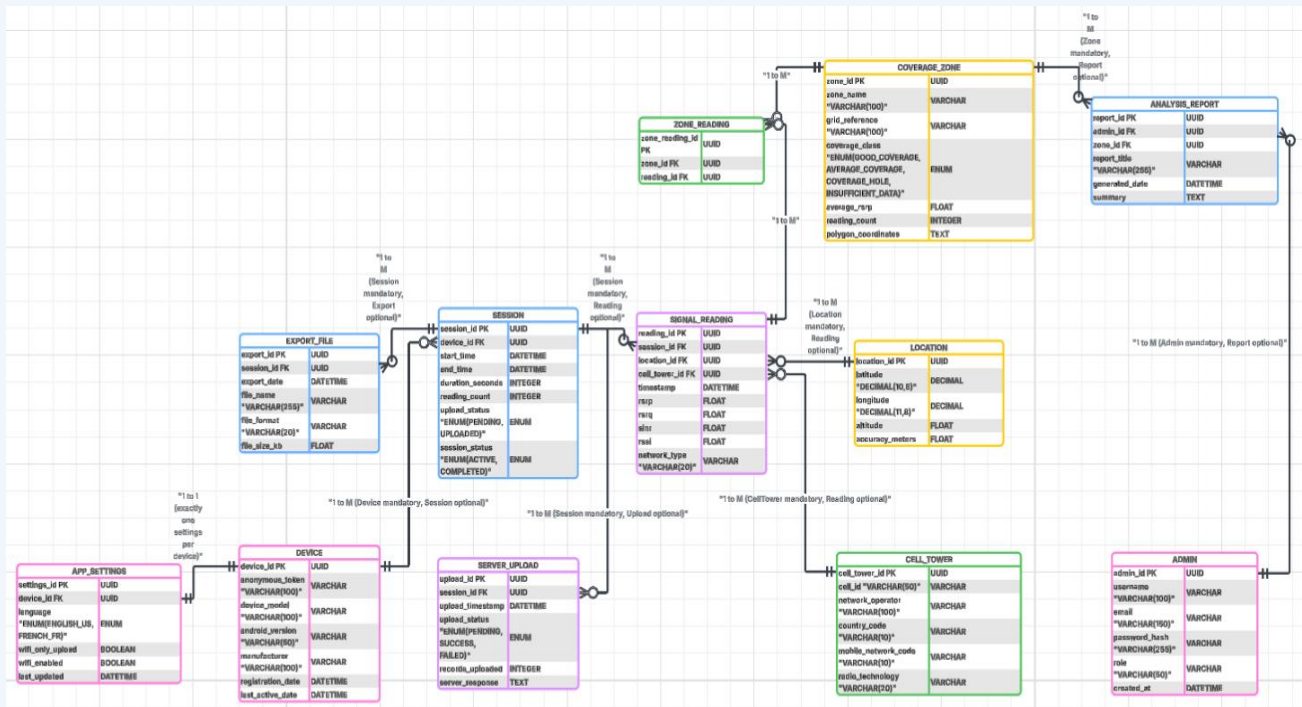


Figure 6.2: Entity Relationship Diagram for CrowdSenseNet

Foreign key constraints are enforced at the database level with cascading deletes where appropriate: deleting a session, for instance, cascades to delete every signal reading, export file, and upload record that belongs to it. A unique constraint on the `device_id` column in `AppSettings` enforces the one-to-one relationship between a device and its settings, and a unique constraint on the `Admin` table's `email` column prevents duplicate administrator accounts.

6.4 Backend Implementation

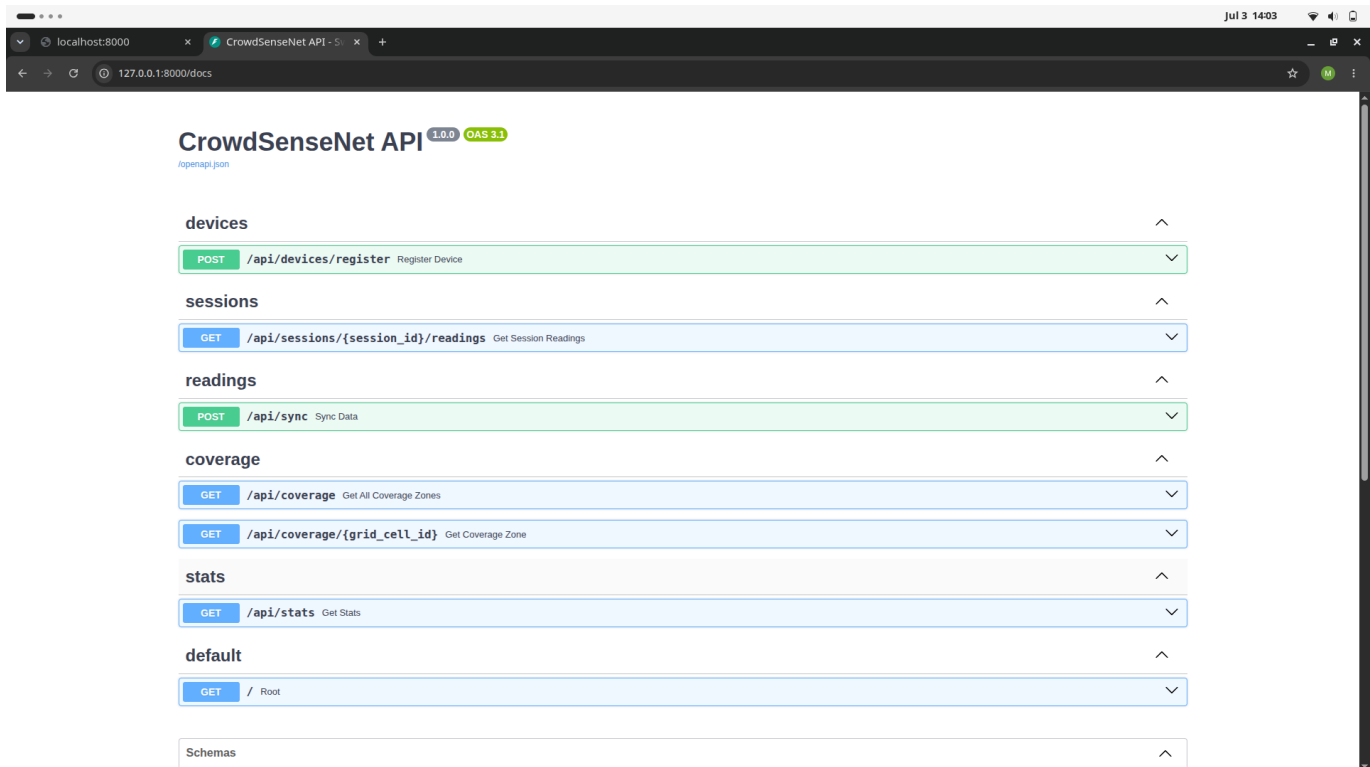
The backend exposes a RESTful API over five endpoint groups: sessions, signal readings, coverage analysis, exports, and administration.

Method	Endpoint	Purpose
POST	/api/sessions	Create a new sensing session record
POST	/api/readings	Upload a batch of signal readings for a session
GET	/api/coverage	Retrieve all coverage zones as GeoJSON
GET	/api/export/:sessionId	Export session data as CSV or GeoJSON
POST	/api/admin/login	Authenticate an administrator and issue an access token

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Method	Endpoint	Purpose
GET	/api/admin/reports	List analysis reports generated by administrators

Table 6.4: Selected CrowdSenseNet REST API endpoints.



A business logic layer sits between the API controllers and the repository layer. It validates every incoming record, rejecting anything with an out of range signal value, a malformed device UUID, or a start time that comes after the end time. Once a batch of readings is accepted, a coverage classification service aggregates all readings inside each grid cell; cells with at least fifty readings per square kilometre receive a mean RSRP value and a coverage class, while cells with fewer readings are marked insufficient data. Report and export generation follow the same pattern: a report service computes summary statistics for a zone, and an export service streams a session's readings as CSV rows or GeoJSON point features.

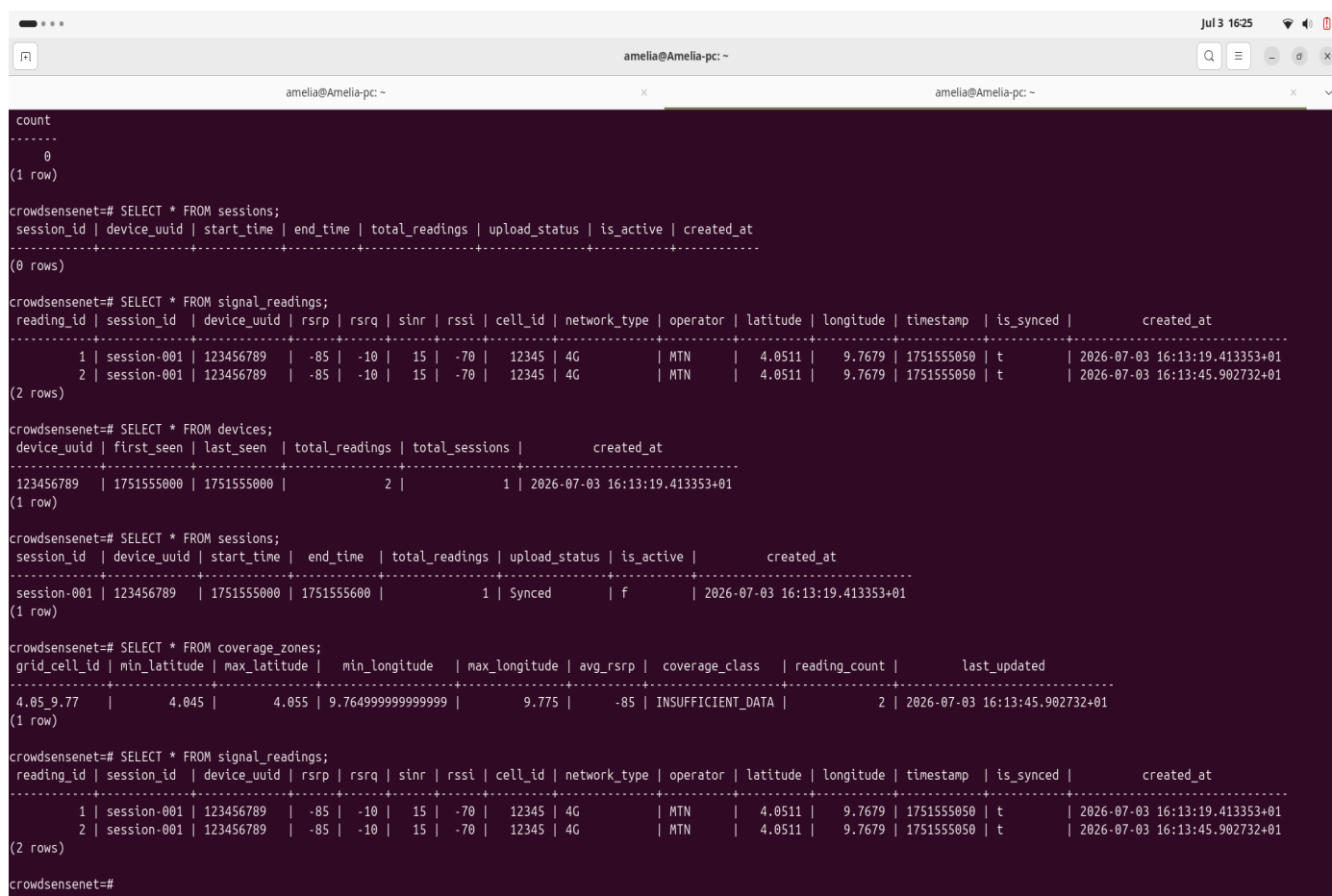
Security follows a privacy first design. Devices identify themselves with the anonymous UUID generated on first launch, included in every API request header, with no account or login required from the end user. Authentication middleware rejects any request lacking a valid, registered UUID. Administrative endpoints carry a second layer of protection, requiring a username and password login that returns a token used on every subsequent dashboard request. The use of the Sequelize ORM provides parameterised queries by default, which closes off SQL injection as an attack path, and all data in transit uses TLS 1.2 or higher.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

6.5 Database Implementation

PostgreSQL was chosen for the server because of its maturity, its full ACID transaction support, and its PostGIS extension for geospatial queries, all of which matter once thousands of geo-tagged readings begin arriving from many devices at once. Room, a thin, compile time checked layer over SQLite, was chosen for the on-device database because it integrates cleanly with Kotlin and Jetpack Compose through Flow and LiveData.

Indexes were added deliberately rather than everywhere: on `device_id` in the `sessions` table, on `session_id` and `timestamp` in the `signal_readings` table, on the composite `latitude` and `longitude` columns in the `locations` table, and on `coverage_class` in the `coverage_zones` table. Each index targets a genuinely frequent query pattern, since every index also adds a small write overhead that is only worth paying where the read benefit is clear.



```
count
-----
      0
(1 row)

crowdsensenet=# SELECT * FROM sessions;
 session_id | device_uuid | start_time | end_time | total_readings | upload_status | is_active | created_at
-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)

crowdsensenet=# SELECT * FROM signal_readings;
 reading_id | session_id | device_uuid | rsrp | rsrq | sinr | rssi | cell_id | network_type | operator | latitude | longitude | timestamp | is_synced | created_at
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
      1 | session-001 | 123456789 | -85 | -10 | 15 | -70 | 12345 | 4G | MTN | 4.0511 | 9.7679 | 1751555050 | t | 2026-07-03 16:13:19.413353+01
      2 | session-001 | 123456789 | -85 | -10 | 15 | -70 | 12345 | 4G | MTN | 4.0511 | 9.7679 | 1751555050 | t | 2026-07-03 16:13:45.902732+01
(2 rows)

crowdsensenet=# SELECT * FROM devices;
 device_uuid | first_seen | last_seen | total_readings | total_sessions | created_at
-----+-----+-----+-----+-----+-----
 123456789 | 1751555000 | 1751555000 | 2 | 1 | 2026-07-03 16:13:19.413353+01
(1 row)

crowdsensenet=# SELECT * FROM sessions;
 session_id | device_uuid | start_time | end_time | total_readings | upload_status | is_active | created_at
-----+-----+-----+-----+-----+-----+-----+-----
 session-001 | 123456789 | 1751555000 | 1751555600 | 1 | Synced | f | 2026-07-03 16:13:19.413353+01
(1 row)

crowdsensenet=# SELECT * FROM coverage_zones;
 grid_cell_id | min_latitude | max_latitude | min_longitude | max_longitude | avg_rsrp | coverage_class | reading_count | last_updated
-----+-----+-----+-----+-----+-----+-----+-----+-----
 4.05_9.77 | 4.045 | 4.055 | 9.764999999999999 | 9.775 | -85 | INSUFFICIENT_DATA | 2 | 2026-07-03 16:13:45.902732+01
(1 row)

crowdsensenet=# SELECT * FROM signal_readings;
 reading_id | session_id | device_uuid | rsrp | rsrq | sinr | rssi | cell_id | network_type | operator | latitude | longitude | timestamp | is_synced | created_at
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
      1 | session-001 | 123456789 | -85 | -10 | 15 | -70 | 12345 | 4G | MTN | 4.0511 | 9.7679 | 1751555050 | t | 2026-07-03 16:13:19.413353+01
      2 | session-001 | 123456789 | -85 | -10 | 15 | -70 | 12345 | 4G | MTN | 4.0511 | 9.7679 | 1751555050 | t | 2026-07-03 16:13:45.902732+01
(2 rows)

crowdsensenet=#
```

6.6 Connecting the Database to the Backend

The Node.js backend talks to PostgreSQL through Sequelize, an object relational mapper that expresses database operations in JavaScript rather than raw SQL, improving readability and closing off most injection risks by default. Connection parameters are read from environment variables rather than hard coded, in line

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

with standard twelve factor application practice, and a connection pool avoids the overhead of opening a fresh connection for every request.

The codebase follows the Repository pattern: a dedicated module for each entity (SessionRepository, SignalRepository, and so on) exposes create, read, update, and delete functions, and the business logic layer calls these functions rather than writing queries directly. This keeps database access centralised, makes the code far easier to unit test, and avoids duplicated query logic scattered across the codebase. Signal readings are inserted in batches using Sequelize's bulkCreate, so a single upload from a device becomes one database transaction rather than hundreds of individual inserts.

```
async function startServer() {
  const app = express();
  const PORT = 3000;

  // 1. Initial middleware setups
  app.use(express.json());
  app.use(express.urlencoded({ extended: true }));

  // CORS-like header support for mobile Retrofit endpoints
  app.use((req, res, next) => {
    res.setHeader("Access-Control-Allow-Origin", "*");
    res.setHeader("Access-Control-Allow-Methods", "GET, POST, OPTIONS");
    res.setHeader("Access-Control-Allow-Headers", "Content-Type, Authorization");
    if (req.method === "OPTIONS") {
      return res.sendStatus(200);
    }
    next();
  });

  console.log("🚀 Mapping CrowdSenseNet RESTful API Endpoints...");
  // 3. Mount Backend API Routes FIRST
  // Map both /api/sessions and /sessions (double-bound for client flexibility)
  app.use("/api", apiRoutes);
  app.use(apiRoutes);

  // 4. Vite Dev Server / Static Injected Production Assets Handler
  if (process.env.NODE_ENV !== "production") {
    console.log("⚙️ Dev Environment: Bootstrapping Vite server middleware...");
    const vite = await createViteServer({
      server: { middlewareMode: true },
      appType: "spa",
    });
    // Let vite handle frontend routes and asset caching
    app.use(vite.middlewares);
  } else {
    console.log("🏠 Production Environment: Serving compiled static build assets...");
    const distPath = path.join(process.cwd(), "dist");

    // Serve build bundle files
    app.use(express.static(distPath));
  }
}
```

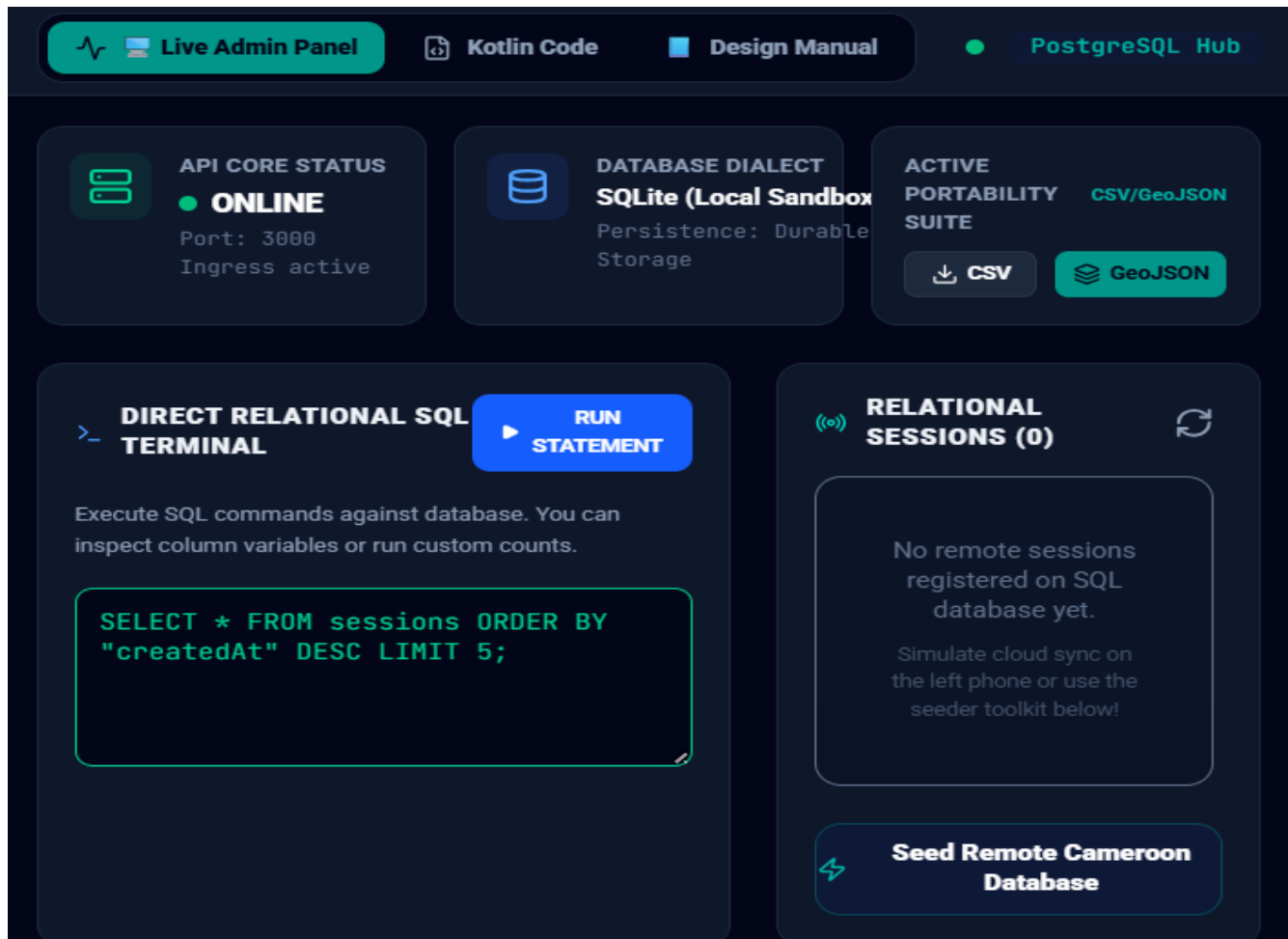
Figure 6.3: Data Synchronisation Sequence

6.7 Testing and Validation

Testing was carried out at three levels. API testing used Postman to exercise every endpoint with both valid and invalid payloads, confirming correct status codes for successful requests, validation failures, and

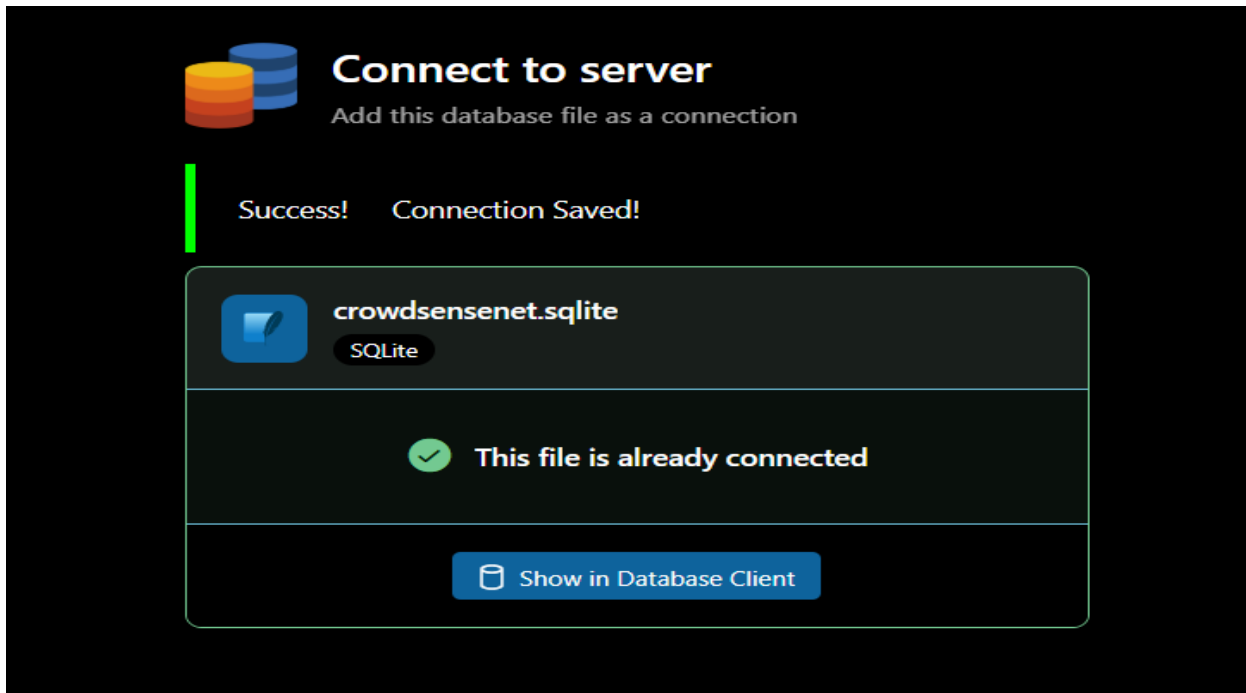
INTERNET PROGRAMMING AND MOBILE PROGRAMMING

missing authentication tokens. Database testing confirmed that foreign key, unique, and not-null constraints are enforced exactly as designed, and reviewed PostgreSQL's query plans to confirm the indexes described above were actually being used rather than sitting idle. Integration testing replicated a full upload payload as the Android app would send it and confirmed that session, reading, and location records all landed correctly in PostgreSQL, with particular attention paid to boundary values: RSRP at exactly -140 and -44 dBm, GPS coordinates at the extremes of the valid range, and sessions with zero duration.



With the backend, database, and their connection tested and working, CrowdSenseNet was functionally complete on the data collection side: real signal readings, collected anonymously from real devices, could now reach a central store and be turned into coverage information. What remained was to build the interface that lets administrators and network engineers actually see and use that data, which is the subject of the next chapter.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING



Chapter 7: The Administrator and Network Engineer Dashboard

Every previous chapter has looked at CrowdSenseNet from the point of view of the person carrying the phone. This chapter closes the loop on the other side of the system: the web based dashboard that lets administrators and network engineers actually see the data that users upload, the moment it arrives, and do something useful with it. Without this piece, CrowdSenseNet would be a data collector with nowhere for its data to go. With it, the system becomes a complete, end to end product: a phone collects a reading, the phone anonymises and uploads it, the backend validates and stores it, and an engineer sitting at a desk in Buea or Douala watches it appear on a live map within seconds.

7.1 Purpose and Position in the System

The dashboard is a browser based web application that sits on top of the same backend server and PostgreSQL database described in Chapter 6. It does not introduce a second data pipeline; it is simply the presentation layer for the data that already exists, built for a different audience and a different device. Where the Android app is designed for a member of the public who wants to install it and forget about it, the dashboard is designed for someone whose job is to look at network coverage all day: a network engineer at MTN or Orange, or a supervising administrator responsible for the health of the CrowdSenseNet platform itself.

Three design goals shaped every screen in the dashboard. First, immediacy: an uploaded reading should be visible to an administrator within seconds of arriving, not after some overnight batch job. Second, trust: because every reading is fully anonymised before it ever leaves a phone, the dashboard has nothing to hide and nothing sensitive to protect beyond the data itself, which keeps the access model simple. Third, actionability: raw numbers on a screen are not useful on their own, so every view in the dashboard is built around a decision an engineer might actually need to make, whether that is spotting a new coverage hole, exporting a dataset for a planning report, or checking whether a batch of readings from a particular area looks trustworthy.

7.2 Architecture Extension

The dashboard is implemented as a single page web application, built with React and communicating with the existing Express REST API described in Section 6.4. It adds one new real time channel to the architecture: a WebSocket connection between the dashboard and the backend server, used specifically so that new uploads appear on an administrator's screen without needing to refresh the page. The moment the backend finishes validating and storing a batch of readings, it publishes a short event over the WebSocket channel, and any dashboard currently open updates its live feed and its map in place.

Layer	Technology	Role
Dashboard frontend	React, with a charting library and a map library	Renders the admin interface in the browser

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Layer	Technology	Role
Real time channel	WebSocket (Socket.IO), layered onto the existing Express server	Pushes new upload events to connected dashboards instantly
Existing REST API	Node.js / Express (Chapter 6)	Serves historical data, exports, and report generation
Existing database	PostgreSQL with PostGIS (Chapter 6)	Single source of truth; no separate admin data store
Authentication	Token based login against the Admin table	Restricts dashboard access to registered administrators

Table 7.1: Technology added to support the administrator dashboard.

Figure 7.1: Extended System Architecture with the Administrator Dashboard

7.3 Administrator Login

Access to the dashboard is restricted to registered administrators, each with a hashed password stored in the Admin table introduced in Chapter 6, and a role of either Super Admin or Analyst. A Super Admin can manage other administrator accounts and change system wide settings such as the coverage hole threshold; an Analyst can view data, generate reports, and export files, but cannot manage accounts. Logging in issues a bearer token that must accompany every subsequent request, and the token expires after a period of inactivity, requiring the administrator to sign in again.

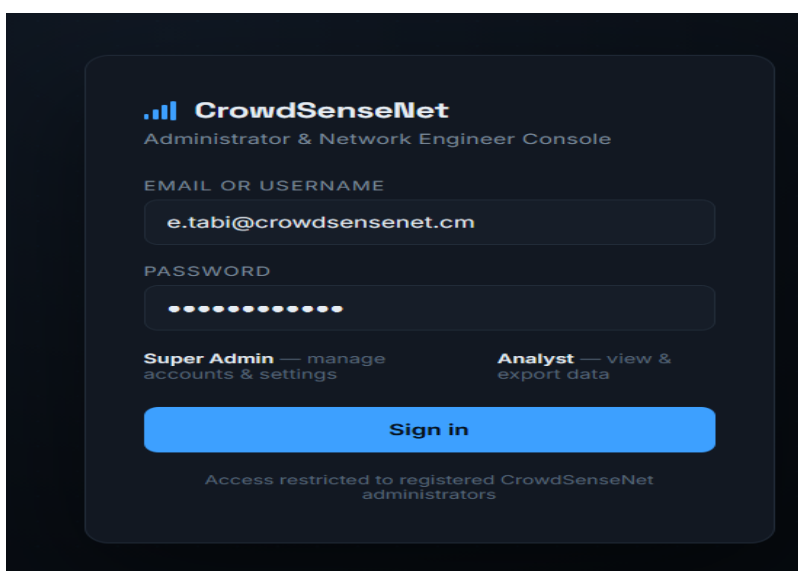


Figure 7.2: Administrator Login Screen

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

7.4 Dashboard Overview

The landing page after login is a summary view designed to answer one question at a glance: is the system healthy and is new data coming in? It shows the total number of registered devices, the number of active sessions in the last twenty four hours, the total reading count for the current day, and the proportion of the country currently covered by at least one reading. Four small trend charts sit below these headline numbers, showing readings received per hour over the last day, the split of readings by network operator, the split of readings by coverage class, and the number of pending versus failed uploads currently queued for retry on the server side.



7.5 Live Data Feed

This is the screen that answers the request at the heart of this chapter: when a user uploads data from their phone, an administrator can see it arrive. The Live Data Feed is a continuously updating table, driven by the WebSocket channel described in Section 7.2, showing each incoming batch of readings as it lands: a timestamp, the anonymous device token, the number of readings in the batch, the network operator detected, the approximate area (a town or district name derived from the coordinates, never an exact address), and a small preview of the average RSRP for that batch. New rows animate gently into place at the top of the table as they arrive, and a running counter in the corner of the screen shows how many readings have been received since the page was opened. Administrators can pause the live stream to inspect a specific batch without losing their place, or apply a quick filter to show only a particular operator or a particular coverage class.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

CrowdSenseNet

Live Data Feed
Uploads appear here the moment they are validated and stored.

Super Admin 12:50:37 WAT

9,569 readings received today All operators All classes Pause stream

TIME RECEIVED	DEVICE TOKEN	READINGS	OPERATOR	APPROX. AREA	AVG. RSRP	CLASS
just now	dev_848ca8cb	16	Nexttel	Buea - Great Soppe	-109 dBm	Average
just now	dev_c440daa6	10	Orange	Buea - Molyko	-89 dBm	Good
just now	dev_6423608e	13	MTN	Garoua - Centre	-98 dBm	Average
just now	dev_bee1d12b	8	Nexttel	Limbe - Mile 4	-113 dBm	Poor
just now	dev_00820f8e	14	Orange	Douala - Bonabéri	-97 dBm	Average
just now	dev_b2911c24	13	Orange	Douala - Bonabéri	-123 dBm	Poor
just now	dev_79804d16	9	Nexttel	Bamenda - Old Town	-93 dBm	Good
just now	dev_d468ca34	6	MTN	Buea - Great Soppe	-106 dBm	Average
just now	dev_986ff76c	8	Nexttel	Yaoundé - Mfoundi	-88 dBm	Good
just now	dev_10630149	19	MTN	Douala - Akwa	-92 dBm	Good
just now	dev_c9849594	10	Nexttel	Bamenda - Nkwen	-120 dBm	Poor
just now	dev_e31b64e4	11	Orange	Douala - Akwa	-85 dBm	Good
just now	dev_148eba8e	10	Nexttel	Bamenda - Nkwen	-122 dBm	Poor

Figure 7.4: Live Data Feed Screen

Clicking any row in the live feed opens a detail panel for that specific upload batch, listing every individual reading it contained: RSRP, RSRQ, SINR, RSSI, cell ID, network type, and the exact coordinates and GPS accuracy recorded for each one. This is the point where the raw contribution from a single user's phone becomes fully visible to the people who can act on it, while remaining completely anonymous, since nothing in the record ties it back to a name, a phone number, or any other personal identifier.

7.6 Session and Reading Explorer

Where the Live Data Feed is built for watching data arrive in real time, the Session and Reading Explorer is built for searching through everything that has already been collected. Administrators can filter by date range, network operator, coverage class, geographic region, and minimum reading count, and the results can be sorted by any column and paged through without loading the entire dataset into the browser at once. This screen is where an engineer would go to answer a specific question, such as how many readings exist for a particular district in the last month, or how the average RSRP for MTN in a given area has changed over the last quarter.

Session & Reading Explorer
Search and filter everything collected so far.

Super Admin 12:51:11 WAT

FILTERS

Date range
01 Jun 2026 – 04 Jul 2026

Operator
All operators
MTN
Orange
Camtel
Nexttel

Minimum readings
10

Apply filters

SESSION ID	REGION	OPERATOR	READINGS	AVG RSRP	CLASS	LAST UPDATED
ses_30000	Centre	Orange	209	-107 dBm	Average	11 Jun 2026
ses_30001	North	Camtel	288	-104 dBm	Average	05 Jun 2026
ses_30002	Centre	Nexttel	291	-88 dBm	Good	04 Jun 2026
ses_30003	Southwest	MTN	231	-103 dBm	Average	14 Jun 2026
ses_30004	West	MTN	137	-93 dBm	Good	02 Jun 2026
ses_30005	West	Nexttel	171	-83 dBm	Good	04 Jun 2026
ses_30006	Southwest	Camtel	229	-117 dBm	Poor	23 Jun 2026
ses_30007	West	MTN	45	-111 dBm	Poor	27 Jun 2026
ses_30008	West	Camtel	82	-102 dBm	Average	20 Jun 2026
ses_30009	Northwest	Nexttel	143	-120 dBm	Poor	07 Jun 2026

Showing 1-10 of 238 sessions

1 2 3 4 5

Figure 7.5: Session and Reading Explorer Screen

7.7 Coverage Map and Heatmap

The administrator version of the coverage map is a superset of the map the end user sees inside the Android app, described in Section 5.2. It renders the same colour coded coverage zones (green for good, amber for average, red for coverage holes, grey for insufficient data) over a real base map, but adds controls aimed specifically at an engineering audience: the ability to switch the underlying metric from RSRP to RSRQ or SINR, the ability to filter the map to a single operator, a time slider to see how coverage in an area has changed over recent weeks, and the ability to click any zone to open the same detail panel described in Section 7.5 for every reading that contributed to it.

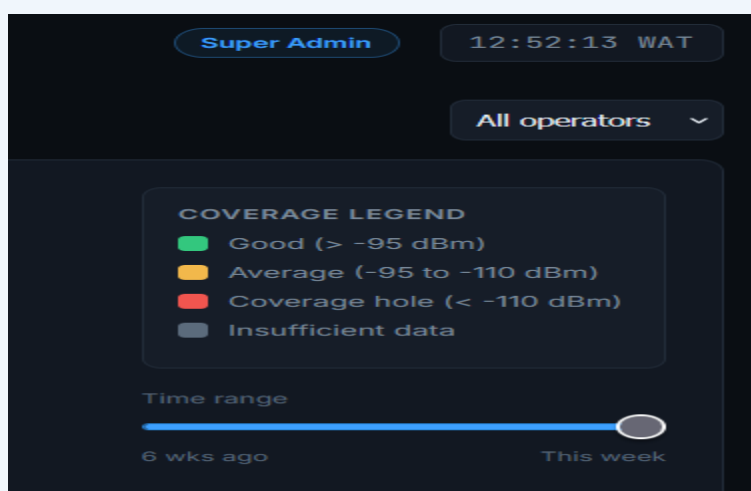


Figure 7.6: Administrator Coverage Heatmap

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

7.8 Device and Session Management

This screen gives administrators visibility into the health of the crowdsensing network itself, rather than the coverage data it produces. It lists every registered anonymous device, showing how many sessions it has contributed, its most recent activity date, its device model and Android version (collected purely to understand whether reading quality varies by hardware, never to identify a person), and its current upload status. Administrators can flag a device if its readings look statistically unusual, for example a device consistently reporting values far outside the plausible RSRP range, which triggers those readings to be excluded from coverage zone calculations until a Super Admin reviews and confirms or clears the flag. No administrator action in this screen can ever reveal who owns a device, since that information was never collected in the first place.

DEVICE TOKEN	SESSIONS	LAST ACTIVE	MODEL	ANDROID	UPLOAD STATUS	REVIEW
dev_1e71c70e	10	49h ago	Samsung Galaxy A05	12	Synced	⚙
dev_6b36bcb8	43	53h ago	Samsung Galaxy A05	11	Synced	⚙
dev_6ebd8294	55	50h ago	Tecno Camon 20	12	Synced	⚙
dev_316de29a	36	29h ago	Infinix Hot 30	12	Synced	⚙
dev_0f73395f	27	2h ago	Infinix Hot 30	11	Pending	⚙
dev_f8f4ab2f	4	37h ago	Samsung Galaxy A14	14	Synced	⚙
dev_6230e342	42	21h ago	Samsung Galaxy A05	11	Synced	⚙

Figure 7.7: Device and Session Management Screen

7.9 Reports and Export

Administrators can generate a structured analysis report for any coverage zone or region, pulling together total reading count, average RSRP, the distribution across coverage classes, and a trend line over a chosen period, and save it against their account for later reference. The same screen exposes the CSV and GeoJSON export functions described in Section 6.4, letting an engineer download a dataset for a specific session, device, region, or date range for use in external tools such as QGIS, without needing to write a single line of SQL against the production database.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

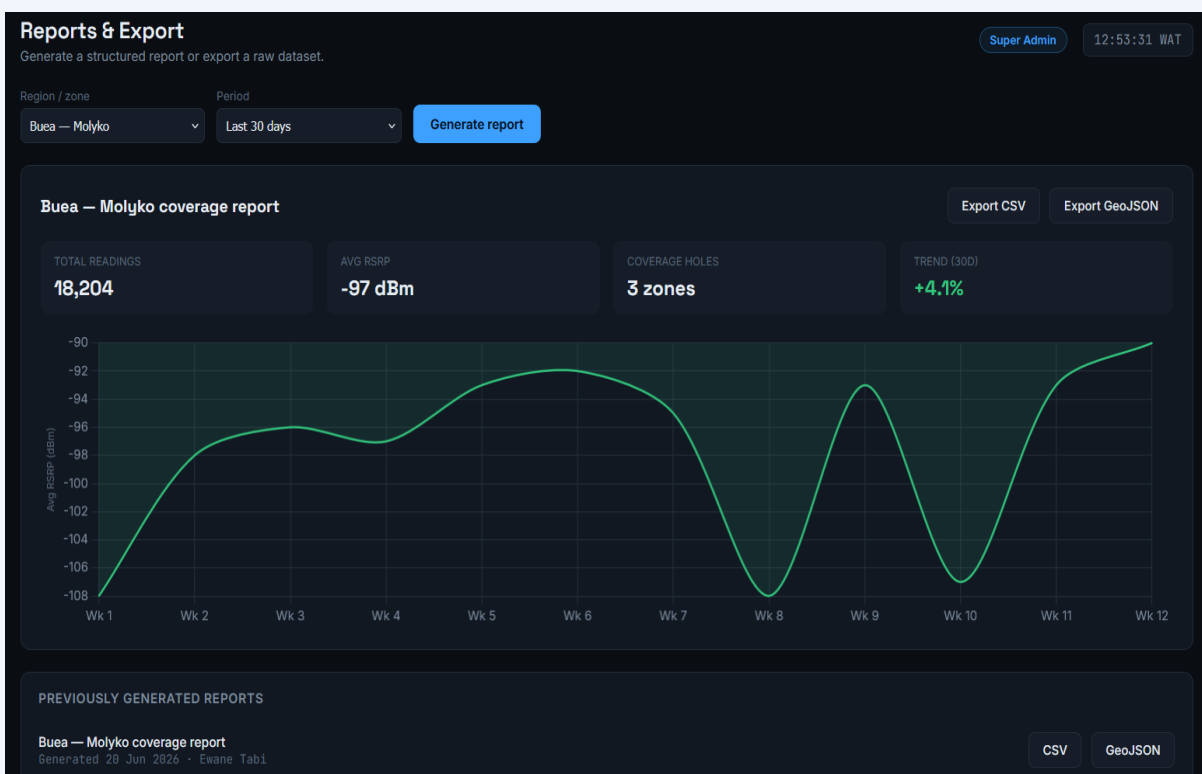


Figure 7.8: Reports and Export Screen

7.10 Data Flow: From a User's Phone to the Admin Screen

It is worth tracing the complete path a single reading takes, end to end, since this is the exact workflow the dashboard was built to support.

- A user is carrying their phone with CrowdSenseNet running in the background. The app reads RSRP, RSRQ, SINR, and RSSI from the TelephonyManager API and a GPS fix from FusedLocationProvider, discards the reading if the GPS accuracy is worse than fifteen metres, strips out anything that could identify the device beyond its anonymous UUID, and writes the reading to the local Room database.
- When the phone is next connected to WiFi (or immediately, if the user has enabled mobile data upload in Settings), the app batches its unsynced readings and sends them to the backend over an authenticated HTTPS request.
- The backend validates the batch, checks every coordinate falls inside Cameroon's bounding box, averages any near duplicate readings, and writes the accepted records to PostgreSQL inside a single transaction.
- The instant that transaction completes, the backend publishes an event on the WebSocket channel described in Section 7.2.
- Any administrator with the dashboard open sees the new batch appear at the top of the Live Data Feed within a second or two, and the Coverage Map updates the relevant zone the next time its aggregate statistics are recalculated.

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

This is what makes CrowdSenseNet a genuinely complete, functional application rather than a mobile prototype with an unfinished back office: the same event that means something to an ordinary user, my phone just contributed to the map, means something immediately and visibly to the engineer whose job is to act on that map.

7.11 Access Control and Privacy Safeguards

Because the dashboard exposes real, aggregated network data to a professional audience, its access model was kept deliberately strict even though the underlying data is anonymous. Every dashboard request requires a valid administrator token, every administrative action (flagging a device, generating a report, changing a system setting) is written to an audit log with the acting administrator's ID and a timestamp, and the Super Admin and Analyst roles introduced in Section 7.3 ensure that only a small number of trusted accounts can change how the system behaves rather than simply viewing it. At no point does any screen in the dashboard expose a name, a phone number, an IMEI, or any other personally identifying information, because none of that information exists anywhere in the database to begin with. Privacy by design, established as a non-negotiable requirement in Chapter 3, holds all the way through to the very last screen in the system.

Conclusion

This report has traced CrowdSenseNet from a research question to a working, end to end system. It began with a broad review of how mobile applications are typically built, then narrowed to a specific, evidence based understanding of a real problem: mobile network coverage in Cameroon is inconsistent, traditional drive testing cannot reach every place that problem shows up, and ordinary smartphones already carry everything needed to help close that gap.

Every major decision in the system traces back to real evidence rather than assumption. The choice to make the app fully anonymous came from the fifty six survey respondents who made it a hard condition of participation. The choice to pause data collection when a phone is stationary came from the direct conflict between users wanting continuous monitoring and users wanting minimal battery drain. The fifteen metre GPS accuracy threshold and the minus one hundred and ten dBm coverage hole definition both came directly from network engineers who work with these numbers every day. Nothing in the final design was arbitrary.

The result is a system with three working parts that fit together cleanly. The Android application, described in Chapters 4 and 5, collects signal data passively and respectfully, giving the user full visibility and control over what is happening on their own device. The backend and database, described in Chapter 6, receive that data, validate it, and turn raw readings into classified coverage zones. The administrator dashboard, described in Chapter 7, closes the loop by giving the people who can actually act on the data, network engineers and researchers, a live, trustworthy window into what users across Cameroon are experiencing on their phones right now.

Some parts of the original vision remain deliberately out of scope for this phase. Machine learning based coverage prediction, multi-operator comparative analytics, and iOS support were all identified early as valuable but were consciously deferred, each for a specific, documented reason, rather than left as an oversight. The architecture, the database schema, and the data pipeline were all built with these future additions in mind, so extending the system later should not require rebuilding what already exists.

Taken as a whole, this project demonstrates that a small, evidence driven team can design and build a crowdsensing platform that is technically sound, genuinely respectful of the people contributing to it, and immediately useful to the organisations that would operate it. It is a credible, lower cost alternative to a small part of what traditional drive testing does today, and a foundation that can grow well beyond that starting point.

References

- Android Developers. TelephonyManager API Reference. Android Developer Documentation. developer.android.com/reference/android/telephony/TelephonyManager
- Android Developers. Room Persistence Library. Android Developer Documentation. developer.android.com/training/data-storage/room
- PostgreSQL Global Development Group. PostgreSQL 16 Documentation. postgresql.org/docs/
- Sequelize. Sequelize ORM Documentation. sequelize.org/docs/v6/
- Express.js. Express.js API Reference Documentation. expressjs.com/en/4x/api.html
- 3GPP. TS 36.214: Evolved Universal Terrestrial Radio Access (E-UTRA); Physical Layer; Measurements. 3rd Generation Partnership Project.
- Codd, E. F. Further Normalisation of the Data Base Relational Model. IBM Research Report RJ909.
- Fowler, M. Patterns of Enterprise Application Architecture. Addison-Wesley.
- Richardson, L., and Ruby, S. RESTful Web Services. O'Reilly Media.
- Martin, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- Flutter. Flutter Documentation. docs.flutter.dev
- React Native. React Native Documentation. reactnative.dev/docs/getting-started
- Ionic. Ionic Framework Documentation. ionicframework.com/docs
- Microsoft. .NET MAUI Documentation. learn.microsoft.com/en-us/dotnet/maui/
- Apple Developer. Swift Programming Language. swift.org/documentation/
- Google Developers. Kotlin for Android. developer.android.com/kotlin/